

Pipeline de projetos de aplicação com IA

Um kit reutilizável para tirar uma aplicação do zero com agentes de IA e sustentá-la até a entrega — com portões objetivos, decisões rastreáveis e orçamento de contexto medido em número.

Tudo o que este documento afirma que o pipeline faz está apontado para o arquivo e a linha onde a coisa existe. Onde a evidência é fraca, isso está escrito. Onde o pipeline não serve, isso também está escrito — nas páginas finais, não nas notas de rodapé.

AUTOR

Gustavo Alves

REPOSITÓRIO

pipeline-projetos-IA

ESCOPO MEDIDO

24 commits · 22/07 a 13/08/2026

DATA

14 de agosto de 2026

§ Como usar este documento

Ele foi montado na ordem exata em que a apresentação deve ser feita. Ler de ponta a ponta é a apresentação inteira; ler só as Partes I, VI e VIII é a versão de vinte minutos.

A banca tem gente técnica e gente funcional. O documento resolve isso separando duas camadas em toda seção: o **parágrafo de abertura explica a ideia sem jargão** — qualquer pessoa entende — e a **caixa cinza logo abaixo mostra onde aquilo mora no repositório**, com arquivo e linha, para quem quiser conferir. Quem não é técnico pode pular todas as caixas cinzas e não perde nada da história.

EXEMPLO DE PROVA `README.md:25` — a linha citada, literal, para conferência direta no repositório.

As quatro coisas que a apresentação precisa entregar

PERGUNTA DA BANCA	ONDE ESTE DOCUMENTO RESPONDE	PARTE
Para que serve?	O problema real, com o custo já pago em projetos anteriores, e o ciclo de sete passos que ataca cada modo de falha.	I e III
Isso funciona mesmo?	Três fontes de evidência com pesos diferentes: uma medição de uso, uma auditoria adversarial externa e uma avaliação de campo com critério hashado antes dos dados.	V e VI
Quanto custa e quanto economiza?	Os cinco mecanismos de economia, cada um com o número medido — e o imposto que o pipeline cobra de volta, declarado.	VI
Onde isso quebra?	A tabela de limites: os contextos em que o kit deixa de servir, e a cobertura automática real (11% dos itens de checklist).	VIII

A REGRA QUE VALE PARA O DOCUMENTO INTEIRO

Nenhum número aqui foi estimado, arredondado para cima ou inventado. Todos foram contados no repositório ou reproduzidos rodando o código — inclusive os que não favorecem o projeto. Dois erros de documentação foram encontrados durante a preparação deste dossiê e estão registrados na Parte VIII, com a linha exata. Isso não é autocrítica decorativa: é a mesma regra que o pipeline aplica a quem o usa.

§ Sumário

Nove partes, trinta e seis seções. O tempo indicado é o de fala, não o de leitura.

PARTE I · POR QUE ISTO EXISTE — 5 min

1	O problema, com o custo já pago	5
2	O que o pipeline é — e o que ele não é	6
3	O ciclo de sete passos	7

PARTE II · COMO O PIPELINE É FEITO — 8 min

4	As sete regras, e o que cada uma custou para nascer	8
5	O mapa do repositório: onde cada coisa mora	8
6	O vocabulário mínimo	10
7	Quem faz o quê: a IA e o dono	10

PARTE III · O CAMINHO, DO DIA 1 À ENTREGA — 6 min

8	As sete fases e seus portões	12
9	Depois da entrega: sustentar o que já roda	13

PARTE IV · OS 24 AGENTES — 15 min

10	O que é um agente, e por que só um por sessão	15
11	Os seis mecanismos que fazem um agente ser bom	15
12	Agentes de fase — 7 fichas	17
13	Agentes de arquitetura — 2 fichas	20
14	Agentes de backend — 3 fichas	21
15	Agentes de frontend — 2 fichas	23
16	Agentes transversais — 6 fichas	24
17	Agentes de sistema vivo — 3 fichas	27
18	Agente de dados e análise — 1 ficha	29

PARTE V · OS TESTES — 12 min

19	Por que um kit de processo tem testes	31
20	<code>check.py</code> — o portão automático, checagem por checagem	32
21	<code>test_check.py</code> — 57 testes, classe por classe	33
22	As iscas: o portão que testa o portão	35
23	Windows, encoding e o CI em duas plataformas	37
24	A varredura de segredo — e a lacuna que fica declarada	38

PARTE VI · CUSTOS — 10 min

25	Onde o dinheiro vaza num projeto feito com IA	40
26	O que foi medido, e o peso de cada evidência	40
27	Os cinco mecanismos de economia, com o número de cada um	41
28	O imposto: o que o pipeline cobra de volta	43

PARTE VII · OBSIDIAN E OS GRAFOS — 5 min

29	Por que Obsidian, e por que nada depende dele	45
----	---	----

30	O grafo: o que ele mostra que uma lista de pastas não mostra	45
31	O que o vault já vem configurado para fazer	47
PARTE VIII · O QUE EU CONSTRUI — 6 min		
32	A linha do tempo: da v1 à v13.4 em 23 dias	49
33	As correções que só apareceram porque foram medidas	50
34	Onde o pipeline para	51
PARTE IX · FECHO — 8 min		
35	As perguntas que vão ser feitas, com a resposta pronta	53
36	Índice de provas	54

1. O problema, com o custo já pago

Construir software com agentes de IA funciona muito bem por umas duas horas. Depois começam seis coisas — e todas as seis já custaram caro em projetos reais.

Vale começar pela parte que costuma ser mal-entendida: **nenhum destes seis problemas é problema de modelo ruim**. Todos são problemas de processo ausente. Trocar o modelo por um melhor não resolve nenhum deles, e é exatamente por isso que a resposta certa é um processo, não uma ferramenta nova.

O QUE ACONTECE	COMO SE SENTE NA PRÁTICA	O CUSTO JÁ PAGO, MEDIDO
A IA refaz tudo em vez de mexer no trecho certo	"ela consertou o bug e quebrou outras três coisas"	O maior custo isolado de dois projetos
O contexto incha e é relido em toda sessão	as respostas ficam lentas, caras e mais burras	Um "estado atual" de 1.930 caracteres, relido em cada sessão
O estado se espalha por vários documentos e diverge	dois arquivos com números diferentes, ninguém sabe qual vale	Um card dizendo "63 testes" ao lado de outro dizendo "85"
A IA inventa dado para parecer completa	um número plausível que ninguém consegue rastrear	Relatório que mente em silêncio
Decisões são re-discutidas toda semana	"por que a gente não usa microserviços mesmo?" pela quarta vez	Sessões inteiras re-litigando o que já tinha morrido
"Parece bom" passa como aceite	o defeito aparece na frente do cliente	6 ciclos de QA por um parser escrito sem ver a fonte real

PROVA `b_process/f_glossary_and_primer.md:21-28` — a tabela completa dos seis modos de falha, com o custo de cada um. · `docs/a_scb_usage_analysis_260722_0000.md:25-35` — a medição de onde cada número saiu.

Os três que mais doem, em detalhe

1. Regeneração — o custo número um

Você pede para corrigir o cálculo do troco. A resposta certa são as quatro linhas da função `calcular_troco`. A resposta errada é o arquivo `caixa.py` inteiro reescrito "já melhorando umas coisas". O problema não é a IA gostar de escrever: é que o *diff* fica grande demais para alguém revisar de verdade — então ninguém revisa, e código que estava certo e testado volta diferente.

2. Contexto inchado — o custo que se paga toda vez

Um projeto real tinha um "estado atual" de 1.930 caracteres (cerca de 640 tokens) escrito como um parágrafo-parede único, com mais de quinze referências cruzadas. Ele era relido em **toda** sessão. A regra do kit da época dizia " ≤ 1 página" — que foi cumprida na letra e violada no espírito, porque "uma página" não é um limite, é um convite à compressão.

3. Estado espalhado — a divergência que ninguém vê

No mesmo projeto, versão e métricas viviam em quatro lugares ao mesmo tempo. O flagrante: um card do backlog pedindo *rebuild* com "esperado 63 testes" convivendo com o card vizinho dizendo "85 testes". E existiam **dois backlogs** — um na raiz e outro numa subpasta. A checklist da época até alertava contra números conflitantes, mas nenhuma regra atacava a causa, que era a duplicação de estado.

A FRASE QUE RESUME A PARTE I

O pipeline existe para que a IA construa **por delta**, com **portão objetivo**, registrando cada decisão, gastando pouco contexto — e para que o que ela não sabe fique **declarado como lacuna** em vez de virar invenção.

PROVA `b_process/f_glossary_and_primer.md:34-36`

2. O que o pipeline é — e o que ele não é

Começar pelos limites não é modéstia. É o que faz a equipe confiar no resto — e é a única forma de a apresentação sobreviver à primeira pergunta difícil.

O pipeline é um **kit de disciplina com algumas travas automáticas**. Ele cabe numa pasta de documentação que acompanha o projeto, tem 53 arquivos de texto e 6 scripts Python sem nenhuma dependência externa, e funciona em qualquer ferramenta de IA. Ele não é um framework, não é um produto, e não precisa de instalação além de Python.

24

AGENTES INSTALÁVEIS

cada um com regras e
portão próprios

284

ITENS DE CHECKLIST

118 na checklist geral + 166
nas skills

30

JULGADOS POR MÁQUINA

14 reprovam o commit · 16
avisam

57

TESTES DE REGRESSÃO

rodando em Linux e
Windows a cada push

O que ele NÃO é — quatro afirmações que o próprio repositório faz sobre si

NÃO É...

POR QUÊ, COM O NÚMERO

Um sistema que impede erro

São 284 itens de checklist e o script julga 30 deles — cerca de **11%**. O resto depende de alguém rodar a seção certa. Quem afirma que o processo impede erro está vendendo alguma coisa. O que ele faz é tornar o erro *visível e rastreável*.

Uma metodologia de gestão

Não tem estimativa, sprint, atribuição por pessoa nem *velocity*. O backlog é uma lista com limite de trabalho em paralelo — não é um Jira.

Substituto de revisão humana

A revisão adversarial é uma IA atacando o que outra IA construiu. Ajuda muito e não é a mesma coisa que um par humano olhando o código.

Bom para tudo

A tabela de limites (seção 34) lista os contextos em que ele deixa de servir: aplicação com mais de 30 módulos, multi-repo, time acima de quatro pessoas.

PROVA `README.md:25` — "o kit tem **284** itens de checklist (118 no CHECKLIST + 166 nos `b_process/skills/`); scripts/check.py julga **30** deles (14 reprovam o commit, 16 avisam) — cerca de 11%." · Os três números foram recontados no repositório durante a preparação deste dossiê e conferem exatamente.

POR QUE ESSA FRASE EXISTE NO README

Porque ela já envelheceu uma vez. A versão anterior dizia "188 itens / 18 checagens" enquanto o disco tinha 277 e 23. A frase mais honesta do kit era justamente a que mentia em silêncio. A correção não foi trocar os números: foi criar um **teste** que reconta os itens a partir dos arquivos e reprova o commit se a frase do README divergir do disco.

PROVA `scripts/test_check.py:491` — classe `TestHonestidadeDeclarada` . O contador está em `test_check.py:500-516`.

3. O ciclo de sete passos

Se a equipe sair da apresentação sabendo só isto, já valeu. Noventa por cento do dia a dia é este laço.

O CICLO DE UMA SESSÃO

1. ABRIR

uma skill (o papel da sessão) + o contexto-fonte + só o arquivo do momento

2. PEDIR

delta — o trecho que muda, nunca "refaz tudo"

3. PASSAR

pelo portão — a checagem objetiva, rodada na SUA máquina

4. REGISTRAR

decisão → D-NN · bug → QA-NN · pendência sua → Q-NN

5. ATUALIZAR

o "Estado atual" do contexto, por substituição

6. DATAR

a linha no changelog

7. COMMITAR

citando os IDs

PROVA [b_process/f_glossary_and_primer.md:59-67](#) · o mesmo ciclo aparece em [README.md:93](#) e [b_process/a_roadmap.md:142-149](#), sempre apontando para a mesma fonte — nunca reescrito.

Os passos 5 e 6 são os que mais se pula, e são exatamente os que fazem o estado divergir e o histórico sumir. Por isso existe um *template* de fecho de sessão no Obsidian: o fecho vira um clique, e o que faltou fica visível na lista.

Cada passo ataca um modo de falha específico

PASSO	ATACA	COBRADO AUTOMATICAMENTE?
1 · Abrir só o necessário	contexto inchado	Sim — orçamento de 4.000 caracteres é reprovação de commit
2 · Pedir delta	regeneração	Não — é disciplina do dono
3 · Passar no portão	"parece bom" como aceite	Parcialmente — a metade formal é máquina
4 · Registrar D/Q/QA	decisão re-litigada, bug que volta	Sim — ID citado que não existe reprova o commit
5 · Atualizar por substituição	estado espalhado	Sim — divergência entre backlog e contexto reprova
6 · Datar no changelog	histórico dentro do contexto	Aviso — sessão sem skill declarada
7 · Commitar citando IDs	correção sem rastro	Sim — ID inexistente reprova

4. As sete regras, e o que cada uma custou para nascer

Nenhuma delas é uma boa-prática genérica copiada de um blog. Cada uma existe porque um projeto real pagou por não tê-la — e o repositório guarda qual foi a conta.

#	A REGRA	POR QUÊ — O CUSTO QUE A GEROU
1	Contexto com orçamento em número. O contexto-fonte tem ≤ 4.000 caracteres, medível por script, atualizado por substituição.	"≤ 1 página" sem número virou, num projeto real, um parágrafo-parede de ~640 tokens relido em toda sessão. Adjetivo não é limite.
2	Histórico fora do contexto. O que é datado vai para o changelog, que nenhuma sessão carrega.	173 linhas de histórico que nenhuma sessão pagou. Funcionou como desenhado desde a primeira versão.
3	Delta, nunca regenerar. Só o trecho alterado — em documentação e em código.	Regeneração foi o maior custo isolado dos projetos anteriores.
4	Decisão rastreável. Assunto fechado vira D-NN em duas frases; bug vira QA-NN citado no commit; pendência do dono vira Q-NN.	Sem isso, toda sessão re-discute o que já foi decidido. O script cobra: ID citado tem de existir, e ID não se repete.
5	Nada entra sem portão. Critério de aceite objetivo escrito no dia 1. "Parece bom" não passa.	Rejeição registrada vale tanto quanto adoção — a lista de rejeitados é o que impede a re-proposta mensal do que já morreu.
6	Estado mora num lugar só. Versões, métricas e contagens vigentes só no contexto-fonte; todo outro documento aponta.	Num projeto real o estado vivia em 4 arquivos e divergiu de verdade. O script compara "Em andamento" entre backlog e contexto e reprova a divergência.
7	Observe antes de construir. Nenhum parser, integração ou schema sem uma amostra real da estrutura na mão.	Chutar a estrutura de uma fonte custou 6 ciclos de QA num projeto e 6 versões de schema em 7 dias em outro.

PROVA [README.md:82-88](#) — as sete regras com o "por quê" de cada uma. O README é declaradamente o único arquivo do kit que carrega justificativa; os arquivos que os agentes carregam são só instrução, para não pagar contexto com prosa.

DETALHE DE DESENHO QUE VALE MENCIONAR NA BANCA

As justificativas moram todas em **um único arquivo** — o README — e nenhum agente o carrega. Isso não é organização estética: é economia. Explicar "por que" custa tokens em toda sessão se a explicação morar dentro do arquivo que toda sessão lê. A instrução vai para onde o agente lê; o motivo vai para onde o humano lê.

5. O mapa do repositório: onde cada coisa mora

O kit é a pasta de documentação do padrão. O script `new_project.py` o instala num projeto novo como `77777777_<TAG>_Project_DOCs/` e monta o esqueleto de código em volta.

ESTRUTURA, COM A FUNÇÃO DE CADA PASTA

INDEX.md	mapa de navegação — a nota-casa do vault
README.md	os porquês e os limites (o único lugar com justificativa)
CLAUDE.md	contrato de leitura do agente — a ferramenta carrega sozinha
a_context/	A VERDADE
a_context_source.md	contexto-fonte, ≤4.000 chars — o único que TODA sessão carrega
b_plan.md	módulos, contratos, milestones (congela após aprovação)
c_decisions.md	D-NN (2 frases + link) · Q-NN (dono) · QA-NN · arquivamento
<tema>.md	domínio por tema (nasce vazio, leitura sob demanda)
b_process/	COMO SE TRABALHA
a_roadmap.md	o caminho do dia 1 à entrega, fase por fase
b_checklist.md	portões por tipo de entrega — 118 itens
c_backlog.md	fonte única de tarefas, com WIP declarado
d_agent_learnings.md	lições vivas, já com as herdadas
e_repository_standard.md	o padrão de nomes e estrutura
skills/	os 24 agentes (SKILL.md instalável) — 166 itens de portão
profiles/	web-nextjs · dados-python · genérico
templates/	modelos de D-NN, QA-NN e fecho de sessão
c_technical_docs/	guia do Obsidian · caso de referência
d_history/	changelog datado (nenhuma sessão carrega)
e_qa/	relatórios de QA do PROJETO, com timestamp no nome
docs/	auditoria do próprio KIT — não vai para projetos novos
.github/workflows	portão rodando em Linux e Windows a cada push
scripts/	task.py · check.py · test_check.py · install_hook.py new_project.py · arquivar.py

PROVA README.md:43-76 · a contagem de arquivos foi refeita: 53 arquivos .md e 6 arquivos .py, fora .git.

Os seis scripts, e o que cada um resolve

SCRIPT	LINHAS PARA QUÊ
task.py	80 Ponto de entrada único, com 7 tarefas: <code>check</code> · <code>check-all</code> · <code>strict</code> · <code>hook</code> · <code>unhook</code> · <code>test</code> · <code>arquivar</code>
check.py	718 O portão de higiene — 14 checagens que reprovam e 16 que avisam
test_check.py	1.051 Os testes de regressão dos próprios scripts — 57 testes, só biblioteca padrão
new_project.py	482 Cria projeto novo; com <code>--upgrade</code> , atualiza o processo de um existente sem tocar na verdade dele
install_hook.py	139 Instala o <i>pre-commit</i> — sem isso o portão só roda quando alguém lembra
arquivar.py	133 Lista as decisões que nenhum documento vivo cita; só escreve com <code>--aplicar</code>

POR QUE TASK.PY EXISTE, E POR QUE NÃO É UM MAKEFILE

Os comandos do kit moravam em prosa espalhada pelo README, pelo roteiro e pelo guia do Obsidian. Prosa duplicada derrapa: uma auditoria encontrou **três divergências entre documentação e código num único arquivo** — e a regra do próprio kit diz que divergência entre doc e código é achado de QA. Comando que mora em um lugar só não derrapa. Não é `make` porque `make` não existe num Windows por padrão, e o kit não tem nenhuma dependência externa: roda em qualquer máquina com Python, sem `pip install` e sem rede.

6. O vocabulário mínimo

Sete termos. Sem eles, o resto da apresentação é ruído; com eles, a banca acompanha qualquer seção seguinte.

TERMO	O QUE É	O QUE CUSTA IGNORAR
Delta	A IA devolve só o trecho alterado, nunca o arquivo inteiro regenerado.	O diff fica grande demais para revisar, então ninguém revisa — e código que estava certo volta diferente.
Portão (gate)	O critério objetivo que decide se uma entrega é aceita, escrito <i>antes</i> do trabalho começar.	Sem portão, o aceite vira "parece bom", e o defeito é descoberto pelo usuário.
Portão de existência	Um portão especial, no começo de algumas skills, que pergunta se a coisa <i>deveria existir</i> . Reprovar é o resultado esperado na maioria das vezes.	Você paga todo o custo operacional de uma arquitetura distribuída sem receber nenhum dos benefícios dela.
Contexto-fonte	O único arquivo que toda sessão carrega, e a única fonte do estado atual: versão, o que está pronto, o que está em andamento, o que está bloqueado.	Se o estado morar em quatro arquivos, os quatro divergem. Aconteceu, está medido.
D-NN · Q-NN · QA-NN	Decisão registrada (append-only, nunca apagada) · questão que só o dono responde · achado de QA com severidade e reprodução.	Sem D-NN, toda sessão re-discute o já decidido. Sem QA-NN, o bug volta sem o rastro de que já foi corrigido.
Lacuna declarada	O que não se sabe fica escrito como não sabido — nunca preenchido com estimativa ou média plausível.	É o defeito mais silencioso que existe: passa em todos os testes, envenena o número, e alguém decide com base nele.
Sandbox × máquina real	O que o agente roda no ambiente dele é indicativo; o portão de verdade roda na máquina do dono.	Você aprova algo que funciona só no ambiente do agente. Processo vivo tem cache e mascara mudança.

PROVA `b_process/f_glossary_and_primer.md:78-278` – cada verbete traz o que é, um exemplo concreto e o que custa ignorar.

UM PRIMO DO "LACUNA DECLARADA" QUE VALE CITAR EM VOZ ALTA

Ausente ≠ zero. Campo que não veio continua nulo — nunca vira `0`, `" "` ou a data de hoje. Este é o defeito que passa em toda a suíte de testes, produz um número plausível e só aparece quando alguém questiona um resultado que já foi usado para decidir. Cinco skills diferentes o citam explicitamente.

7. Quem faz o quê: a IA e o dono

O pipeline separa isso de propósito, porque a maior parte dos acidentes acontece quando as fronteiras ficam confusas.

A IA FAZ	SÓ O DONO FAZ
escreve código e testes, por delta	roda o portão na máquina real
propõe decisões e registra as aprovadas	decide toda Q-NN — regra de negócio e rumo
encontra e reporta defeitos com reprodução	migration em produção, deploy, <code>git push</code>
declara o que não sabe como lacuna	fornece o dado que só ele tem

PROVA [b_process/f_glossary_and_primer.md:287–292](#) · os quatro papéis do dono em [README.md:98–101](#).

Se a IA decidiu uma regra de negócio, alguém errou o processo — não o modelo. Essa frase é a tese da seção, e ela tem consequência prática: cada skill traz, na seção "Limites", a instrução explícita de registrar Q-NN e *parar* diante de ambiguidade de negócio. Não é uma recomendação no README; está dentro do arquivo que o agente carrega.

As seis frases que economizam uma sessão inteira

Cada uma corta um problema específico antes de ele custar caro. A recomendação do repositório é literalmente colar na parede.

"Isso passou no portão? Mostra o número."

"Cadê o D-NN?"

"Me manda só o delta."

"Rodou na minha máquina ou no sandbox?"

"Viu uma amostra real antes de escrever esse parser?"

"Isso muda alguma decisão ou número?"

PROVA [README.md:103](#) · [b_process/f_glossary_and_primer.md:335–340](#)

8. As sete fases e seus portões

O roteiro é o único arquivo que diz *o que fazer em que ordem*. Cada fase declara: qual agente usar, o que você entrega a ele, e o portão que fecha o passo.

A regra que atravessa todas as fases: **uma skill por sessão · sempre por delta · nada avança sem portão objetivo**. E cada sessão assume que a anterior passou no portão — a ordem não é sugestão.

FASE	O QUE ACONTECE	AGENTE	O PORTÃO QUE FECHA
0	Contexto — transformar a descrição crua num documento curto e verdadeiro	<code>context-bootstrap</code> projeto que já existe: <code>existing-project-adoption</code>	O script passa (≤ 4.000 chars) e o dono lê o contexto inteiro e concorda com cada linha
1a	Decidir a forma da arquitetura	<code>architecture-monolith</code> (padrão) ou <code>architecture-microservices</code>	D-01 registrado com a forma escolhida e o gatilho que faria mudar
1b	Gerar o plano: módulos, contratos, milestones	<code>planner</code>	Para cada módulo: "outro agente implementaria isso lendo só o contrato?" Aprovado = congelado como D-NN
1c	Conferir se os artefatos contam a mesma história	<code>artifact-consistency</code> em sessão separada	Zero achados críticos · todo módulo com ao menos uma tarefa · toda restrição inegociável com um portão que a verifica
2	Dados e domínio — schema, invariantes, regra de negócio	<code>backend-domain</code> antes dele, <code>data-analysis</code> se houver fonte externa	Migration roda num banco vazio · invariantes testados inclusive tentando violar direto no banco · transação não deixa efeito parcial
3	Borda, UI e acesso — na ordem acesso → borda → tela	<code>authentication</code> · <code>backend-bff</code> · <code>microservice-sync</code> · <code>frontend-uiux</code>	Cada rota sensível testada sem sessão (página redireciona e API devolve 401) · 4 estados por tela · teste com upstream fora e lento
4a	Testes	<code>testing</code>	Suíte verde na máquina do dono · roda duas vezes com o mesmo resultado · lacunas declaradas
4b	Revisão adversarial	<code>guardrails-review</code> em sessão separada	Relatório registrado em <code>e_qa/</code> · 12 frentes percorridas · cada achado com reprodução · crítico/alto corrigidos com teste de regressão
5	Empacotar e operar	<code>iac-docker-terraform</code> + <code>observability</code> se roda continuamente	<code>up -d</code> num ambiente limpo · derrubar e subir preserva os dados · versão consultável em runtime · nenhum segredo na imagem
6	Entrega	<code>delivery-review</code>	Pacote aberto e conferido (lista de arquivos + peso em MB) · nenhum segredo no histórico · estado numérico só no contexto

PROVA `b_process/a_roadmap.md:19-116` — cada fase com a tabela "Skill / Entrega ao agente / Você recebe / Portão".

Três decisões de desenho no roteiro que merecem ser ditas em voz alta

A Fase 1c existe porque nenhum script julga significado

Uma sessão de quinze minutos, só de leitura, que procura módulo sem tarefa, tarefa sem módulo, restrição inegociável sem portão, critério de aceite sem número, e plano adotando o que o registro de decisões já rejeitou. O roteiro explica por que ela não pode ser automatizada: o script julga **forma** —

orçamento, link, ID, segredo — de modo determinístico; **módulo esquecido e adjetivo sem número são significado**, e nenhum script julga significado.

PROVA `b_process/a_roadmap.md:50`

A Fase 4 tem um critério de saída, porque "repita até zerar" é laço infinito

A instrução é repetir testes ↔ revisão adversarial até o placar de crítico/alto zerar. Mas há uma cláusula de escape: se o placar não cair em três passagens consecutivas, pare o laço de QA e abra sessão de consistência de artefatos ou de planejamento. Achado que reaparece três vezes não é bug — é sintoma de plano errado, e continuar revisando queima sessões atacando o efeito. O sinal é objetivo e sai dos relatórios que a própria Fase 4b já escreve; não depende de o agente se autoavaliar.

PROVA `b_process/a_roadmap.md:92` · a mesma regra está dentro da skill, em `b_process/skills/guardrails-review/SKILL.md:55` — a skill define a condição em que ela própria deixa de ser a resposta.

Sessões separadas não são burocracia — são ponto cego

Duas fases exigem sessão separada de quem construiu: a 1c e a 4b. A razão é a mesma nas duas e está escrita nos dois arquivos: *o mesmo contexto que construiu não enxerga o próprio ponto cego*. Quem escreveu o plano é a última pessoa que deveria julgar se o plano cobre o contexto.

RITMO ESPERADO, PARA CALIBRAR EXPECTATIVA

Um app pequeno leva da ordem de 10 dias de sessões dirigidas: ~1 dia de contexto e plano, ~5 de implementação, ~2 de QA em passagens repetidas, ~1 de empacotamento, ~1 de entrega e ajustes. Projeto pequeno pode rodar o modo curto — fases 0 → 2 → 4 → 6. A regra: *pula-se fase, não se pula regra*. Contexto orçado, delta, D-NN e portão objetivo existem em qualquer tamanho.

PROVA `b_process/a_roadmap.md:155`

9. Depois da entrega: sustentar o que já roda

A Fase 6 fecha a construção, não o projeto. Daqui em diante o roteiro deixa de ser uma linha e vira um gatilho por situação.

Esta parte do kit nasceu de uma observação simples: **um projeto passa muito mais tempo sendo mantido do que sendo criado**, e a versão 4 do kit só tinha agente para a criação. As cinco skills de sistema vivo — adoção, depuração, performance, observabilidade e dependências — existem por causa disso.

SITUAÇÃO	AGENTE	O PORTÃO QUE NÃO SE NEGOCIA
"quebrou", "está errado", "deu erro"	<code>debugging-diagnosis</code>	Reprodução determinística antes de qualquer edição · causa provada por liga/desliga · teste de regressão citando QA-NN
"está lento"	<code>performance</code>	Baseline medido antes · gargalo apontado por profiler, não por intuição · uma mudança por vez
"não sei o que aconteceu ontem"	<code>observability</code>	O dono responde às 3 perguntas sem abrir o código · nada sensível em log
subir biblioteca, CVE reportada	<code>dependencies-supply-chain</code>	Uma atualização por vez, suíte verde entre elas · CVE tratada, mitigada com prova, ou aceita com D-NN
ideia de melhoria depois do baseline	<code>evolution-auditor</code>	Lista-morta percorrida · portão escrito antes do experimento

DUAS REGRAS DE ENCAMINHAMENTO QUE EVITAM SESSÃO DESPERDIÇADA

"Está lento" × "está lento desde ontem". O segundo é depuração, não performance: algo mudou, e achar o quê é mais barato que otimizar o que já estava rápido.

Precisou depurar duas vezes o mesmo sintoma? O que faltou foi observabilidade. A terceira sessão de arqueologia custa mais que instrumentar de uma vez.

PROVA [b_process/a_roadmap.md:131-132](#) · repetido em [b_process/skills/README.md](#), seção "A ordem entre elas importa".

E o fecho de milestone

Ao fechar um marco, uma sessão de [retrospective](#) destila 3 a 7 lições generalizáveis — **incluindo os erros do próprio agente** — no arquivo de aprendizados. Lição que aparecer em dois projetos vira regra do kit, com entrada no changelog do kit. É esse laço que fez o kit sair da v1 e chegar à v13.4; a Parte VIII mostra a trilha.

10. O que é um agente, e por que só um por sessão

Cada agente é um arquivo `SKILL.md` instalável: um papel especializado, com regras próprias e um portão próprio. Não é um *prompt* mais caprichado — é um contrato.

Um agente aqui tem quatro partes, sempre na mesma ordem: **o papel** (quem ele é e o que se espera dele), **o contexto que ele recebe** (o que carregar e nada mais), **as regras numeradas** (o que fazer e o que é proibido) e **o portão** (a lista de verificação que decide se a entrega passa). No topo do arquivo há um bloco com `name` e `description` — e a `description` é o que faz a skill *disparar sozinha* quando a tarefa combina, sem ninguém precisar escolher da lista.

COMO UM AGENTE É USADO, DAS DUAS MANEIRAS

Instalada — copie a pasta `b_process/skills/<nome>/` para o diretório de skills da ferramenta e invoque por nome (ex.: `/backend-dominio`).
A `description` dispara a skill sozinha quando a tarefa combina.

Colada — abra o `SKILL.md` e cole o conteúdo na sessão.
Funciona em qualquer ferramenta de IA, sem instalação.

Em qualquer um dos casos, a sessão recebe:
a skill + o CONTEXT + só o arquivo do momento. Nunca o repositório inteiro.

PROVA `b_process/skills/README.md:8-12`

A regra que não se negocia: uma skill por sessão

Duas skills na mesma sessão são duas responsabilidades disputando o mesmo contexto, e nenhuma das duas feita direito. Isso tem um segundo motivo, econômico: na versão antiga do kit havia uma pasta `prompts/` separada das skills, e você carregava o prompt e a skill na mesma sessão — pagando duas vezes pela mesma instrução. A sobreposição medida entre o prompt de QA e a skill de guardrails era de 27%. Os papéis de fase viraram skills, e a pasta `prompts/` deixou de existir.

PROVA `README.md:79` — "Um mecanismo só."

PERGUNTA PREVISÍVEL DA BANCA: "PRECISO DECORAR OS 24?"

Não. No dia a dia se usa cinco ou seis. Os outros existem para quando o assunto aparece — e é a `description` de cada um que dispara sozinha quando a tarefa combina. Ninguém consulta uma lista de 24 itens; a lista consulta você.

11. Os seis mecanismos que fazem um agente ser bom

Ler os 24 arquivos lado a lado revela que eles não são 24 ideias diferentes. São seis mecanismos recorrentes, recombinações. Esta é a seção mais importante da Parte IV.

1 · Portão de existência — reprovar antes de construir

Algumas skills começam perguntando se a coisa **deveria existir**, e reprovar é o resultado esperado na maior parte das vezes. A skill de microserviços pergunta quantos times publicam independentemente *hoje*; um time só significa reprovado, e a entrega vira um monólito modular bem desenhado. Isso não é a

IA se recusando a trabalhar — é o sistema funcionando. E a linha que faz esse mecanismo funcionar de verdade é a que **já entrega a alternativa junto com a reprovação**, removendo o incentivo de aprovar só para ter o que mostrar.

PROVA Nomeados como tal em `architecture-microservices/SKILL.md:13` e `frontend-mfe/SKILL.md:13`. Outros quatro reprovam a construção com o mesmo efeito sem usar o nome: `data-analysis:21`, `performance:23`, `dependencies-supply-chain:16` (coluna literalmente chamada "Reprova se") e `microservice-sync:17`.

2 · Verificar o artefato de saída, nunca a intenção

Cinco skills exigem olhar *o que saiu*, não o código que deveria produzi-lo: o *bundle* final em vez da configuração, o artefato instalado em vez do manifesto, uma amostra real de log em vez da função que sanitiza, o pacote aberto e listado em vez do `.gitignore`. É o padrão anti-autoengano mais consistente do kit — e o mais fácil de demonstrar numa banca, porque em todos os casos o código "está certo" e a saída não.

PROVA `frontend-mfe:36` (bundle final) · `dependencies-supply-chain:47` (artefato, não manifesto) · `observability:42` e `guardrails-review:18` (amostra real de log) · `delivery-review:26` (pacote aberto e conferido).

3 · Amostra real antes de qualquer parser

Nenhuma integração, schema ou dublê de teste é escrito sem o payload de verdade na mão. Quatro skills carregam essa exigência como pré-requisito de entrada, e uma delas vai além: define o que entregar *no lugar* quando não há amostra — um modo `--investigar` que a busca. Não basta proibir; a skill diz o que fazer.

PROVA `backend-bff:14` · `data-analysis:24` · `microservice-sync:49` · `testing:27` ("dublê inventado testa a sua imaginação").

4 · Número no lugar de adjetivo

"Rápido", "seguro", "intuitivo", "robusto", "escalável" — sem métrica e sem comando que fique verde — não reprovam nada, logo não são critério. O kit substitui adjetivo por número em toda parte: o prior de aprovação de 20–30% do auditor de evolução, o "fluxo crítico fecha em ≤ 3 ações" do frontend, os quatro números obrigatórios antes de otimizar, o teto de 4.000 caracteres.

PROVA `artifact-consistency:44` · `evolution-auditor:8` · `frontend-uiux:15` · `performance:15-23`.

5 · Fronteira negativa declarada — "o que eu não faço"

As 24 skills têm uma seção chamada *Limites (mesmo tendo sido a skill certa)*, com a mesma justificativa: a `description` diz quando **não escolher** a skill; os limites dizem o que ela **não faz** mesmo tendo sido escolhida certo. O repositório chama extrapolação de escopo de "o defeito mais caro deste kit" — e o script *avisa* quando uma `description` nasce sem fronteira negativa.

PROVA Aviso 9 em `scripts/check.py:299` — "description de skill sem fronteira negativa".

6 · A skill sabe quando ela deixa de ser a resposta

O caso mais incomum dos 24: `guardrails-review` define a condição em que ela própria para de servir e nomeia a substituta. Se o placar de achados críticos não cair em três passagens seguidas, o laço se declara esgotado e a recomendação passa a ser consistência de artefatos ou replanejamento — porque achado que reaparece três vezes é sintoma de plano errado, e mais uma passagem ataca o efeito.

COMO APRESENTAR ISTO EM TRINTA SEGUNDOS

"Um agente bom não é o que escreve melhor. É o que **sabe reprov**ar: reprova a própria construção quando ela não deveria existir, reprova a entrega quando a prova está na intenção e não no artefato, e reprova a si mesmo quando deixou de ser a ferramenta certa."

12. Agentes de fase — 7 fichas

Estes conduzem o projeto de uma fase à seguinte. Dois deles são somente leitura, e é isso que os torna úteis.

01 · bootstrap-contexto

[b_process/skills/context-bootstrap/](#)
PARA QUE SERVE

Transforma "quero um app que faz X" num documento curto e verdadeiro que todas as sessões seguintes vão ler. É o arquivo que define o projeto.

GATILHO

Primeiro contato com um projeto **novo**; ou quando o contexto precisa ser atualizado. Fase 0.

O PORTÃO

O script passa — orçamento respeitado, sem placeholder esquecido · o dono leu o contexto inteiro e **concorda com cada linha** (não "parece bom") · critério de aceite é um comando ou uma checagem objetiva, não um adjetivo · restrições da stack preenchidas **antes** de qualquer pedido de código · toda suposição marcada como suposição.

O QUE A TORNA BOA

Teto de 5 perguntas, e um método para escolher quais 5: marque cada área como clara, parcial ou ausente, e gaste as perguntas nas de maior impacto × incerteza — não nas cinco primeiras que ocorrerem. E ofereça a resposta que você recomendaria: *o dono corrige uma recomendação errada mais rápido do que responde uma pergunta aberta*. Isso troca interrogatório por revisão. Tem ainda a regra de propagação: ao fechar um Q-NN, pergunte "o que esta resposta acabou de tornar decidível?" — no caso real citado, uma única resposta sobre plano mensal revelou três lacunas de reembolso que ninguém tinha visto.

NÃO USE PARA

Planejar módulos (é o planejador) nem escrever código — *nenhuma linha, nem exemplo*.

PROVA SKILL.md:14–18 (as 5 perguntas), :25 (propagação), :27–32 (portão), :38–41 (limites)

02 · adocao-projeto-existente

[b_process/skills/existing-project-adoption/](#)
PARA QUE SERVE

Pega um sistema que já roda e produz o contexto que ele nunca teve — lendo o **código**, não o README nem a memória de alguém.

GATILHO

"Herdei esse sistema", "código legado", ou simplesmente: existe código antes de existir contexto. Roda uma vez, antes de tudo.

O PORTÃO

Quatro fatos de baseline antes de começar — o projeto builda? os testes passam? sobe do zero? **o que está em produção é este código?** — e, no fim: **nenhum arquivo de código alterado nesta sessão**, verificável por `git diff` limpo.

O QUE A TORNA BOA

O **portão de zero-edição**, com a razão escrita: *mapear e consertar ao mesmo tempo produz um mapa do que você gostaria que existisse, não do que existe*. A tentação de "já que estou aqui, conserto" é a mais forte que existe num código legado, e esta é a única skill que a bloqueia com um comando objetivo. Duas outras coisas: o código ganha

da documentação por regra (a doc é uma hipótese a verificar, e divergência vira QA-NN); e as decisões já tomadas são registradas como **D-NN [retroativa]** com o gatilho que faria mudar — o que impede a próxima sessão de relitigar o que já está construído e pago.

NÃO USE PARA

Projeto novo; nem para refatorar ou consertar o que você mapeou.

PROVA SKILL.md:10 (a razão), :17–27 (baseline), :25 (produção × HEAD), :34 (retroativas), :40 (declarar a cobertura do mapa), :50 e :57 (zero edição)

03 · planejador

b_process/skills/planner/

PARA QUE SERVE

Transforma o contexto num plano curto que pode ser **congelado**: quais módulos existem, o que cada um recebe e entrega, em que ordem construir e qual checagem aprova cada um.

GATILHO

"Por onde começo?", depois que o contexto existe e a forma da arquitetura já foi decidida.

O PORTÃO

Para cada módulo, o dono consegue responder "outro agente implementaria isso lendo só o contrato?" · todo módulo com portão objetivo escrito · milestones em cadeia, cada uma com condição de abertura e portão de saída · nenhum número de estado duplicado do contexto · aprovado = **congelado** como D-NN.

O QUE A TORNA BOA

O teste de autossuficiência do contrato é um critério falsificável para algo normalmente subjetivo. E há uma linha que é a tese do kit inteiro condensada: o ID **M<N>** tem de aparecer nos dois lados — no plano e na tarefa — porque "é o que permite ao script responder sozinho 'existe módulo que ninguém vai construir?'. Sem o ID nos dois lados, essa pergunta vira julgamento humano — e julgamento humano não roda em todo commit." Uma convenção aparentemente burocrática vira cobertura de máquina.

NÃO USE PARA

Decidir a forma da arquitetura, nem implementar — *nenhuma linha*.

PROVA SKILL.md:14 (critério de qualidade), :21 (o ID nos dois lados), :22 (fonte obrigatória para afirmação técnica), :26–30 (portão), :36–39 (limites)

04 · consistencia-artefatos

b_process/skills/artifact-consistency/

PARA QUE SERVE

Confere se os quatro documentos do projeto contam a mesma história antes de alguém escrever a primeira linha de código. Acha o módulo que ninguém vai construir, o critério que é só um adjetivo, e o plano que adota o que já foi rejeitado.

GATILHO

A frase "posso começar a implementar?". É o último portão antes do primeiro código. **Sessão separada** de quem escreveu o plano.

O PORTÃO

Zero achados críticos em aberto · todo módulo do plano com ao menos uma tarefa (ou declarado fora do v1) · toda restrição inegociável com um portão que a verifica · todo critério de aceite é um comando ou um número, não um adjetivo.

O QUE A TORNA BOA

Ela divide o trabalho com o script de forma explícita: rode o **check.py** antes e gaste a passagem humana no que ele não vê, porque "marcação existe é diferente de marcação correta". Isso é o oposto de duplicar trabalho de máquina. E blinda a restrição inegociável contra reinterpretação: "violá-la nunca é MÉDIO, e a saída é ajustar o plano — nunca reinterpretar a restrição para ela caber."

NÃO USE PARA

Caçar bug em código; nem para escrever ou corrigir os documentos — a sessão é somente leitura, inclusive para "melhorar a redação".

PROVA SKILL.md:11–13 (por que sessão separada), :15–17 e :36–39 (divisão com o script), :44 (adjetivo), :57–58 (restrição), :91–96 (portão)

05 · auditor-evolucao

b_process/skills/evolution-auditor/

PARA QUE SERVE

Avalia ideias de melhoria com ceticismo deliberado e aprova poucas. Matar uma ideia ruim vale mais do que vender uma ideia bonita.

GATILHO

O sistema já funciona e alguém pergunta "o que a gente melhora agora?".

O PORTÃO

Prior de aprovação: 20–30% · lista-morta percorrida e citada explicitamente · cada proposta com portão escrito antes de qualquer experimento · cada proposta com probabilidade de passar justificada, não chutada · rejeições registradas como D-NN com o motivo que matou.

O QUE A TORNA BOA

O **prior numérico**. "Seja cético" não muda comportamento de ninguém; "aprove 2 ou 3 de cada 10, e justifique se subir" muda. A lista-morta impede o ciclo de re-litígio que consome equipes — e por isso esta é a única fase que carrega o registro de decisões inteiro. E ela fecha a saída favorita de toda avaliação: *evidência insuficiente = reprova por falta de dado*, nunca "provavelmente ajuda". Adoção obriga a declarar o custo completo — rebuild, bump de versão, migração, retrabalho de documentação —, porque *adoção com custo escondido é rejeição adiada*.

NÃO USE PARA

Antes de existir baseline funcionando; nem para implementar o que foi aprovado.

PROVA SKILL.md:8 (prior), :14 (lista-morta), :16 (portão antes do experimento), :20 (custo da adoção), :24–28 (portão), :34–36 (limites)

06 · revisao-entrega

b_process/skills/delivery-review/

PARA QUE SERVE

Confere o que vai sair pela porta: nenhum segredo, nenhum lixo, tamanho certo, documentos que não se contradizem. Não olha a lógica — olha o pacote.

GATILHO

"Vamos empacotar", "gera o zip", "manda para o cliente". Fase 6.

O PORTÃO

`check.py --historico-completo verde` · nenhum segredo na árvore **nem no histórico** · amostra de segredo em documentação **inutilizada** (xxxx), não apenas isenta · pacote gerado, **aberto e conferido**, com lista de arquivos e peso em MB · nenhum estado numérico duplicado fora do contexto · achados críticos/altos zerados.

O QUE A TORNA BOA

A distinção mais fina do kit inteiro: **isentar um segredo de exemplo com `checar:ignore` cala o seu scanner, não o de quem recebe o pacote**. A exigência é inutilizar o valor, não silenciar o alarme — a diferença entre resolver o problema e esconder o aviso. Esta lição não é teórica: a primeira versão do laudo de auditoria do kit trazia iscas intactas marcadas com `checar:ignore`, e o GitHub barrou o push. Corretamente. Além disso, a skill trata o próprio scanner como falível: *"a varredura do script é rede de arrasto, não garantia."*

NÃO USE PARA

Caçar bug de lógica; nem alterar código. Se algo está errado, é achado, não conserto. E não libera com achado crítico aberto, mesmo que o pacote esteja limpo.

PROVA SKILL.md:14 (rede de arrasto + a flag), :17 (estado duplicado é o defeito nº 1 da fase), :23-29 (portão), :25 (inutilizar × isentar), :35-37 (limites)

07 · retrospectiva

b_process/skills/retrospective/

PARA QUE SERVE

Destila o que esta leva de trabalho ensinou em 3 a 7 lições que servem para os próximos projetos — incluindo, obrigatoriamente, os erros do próprio agente.

GATILHO

Fechar um milestone ou concluir uma entrega.

O PORTÃO

Pelo menos uma lição é **um erro do próprio agente** · nenhuma lição é anedota presa ao projeto · nenhuma duplica linha já existente · **a frase final diz algo que o dono não gostaria de ouvir, mas precisa.**

O QUE A TORNA BOA

Duas coisas raras. Primeira: um **portão que exige autocritica da IA** — um modelo otimizado para agradar não produz isso espontaneamente, e o portão torna a ausência disso uma reprovação. Segunda: um **filtro de admissão com quatro critérios** (incomum · opinativo · tribal · consistente) e exemplos trabalhados dos dois lados: *"Testar antes de entregar" falha em incomum e em tribal: é redescritção do óbvio. "Sandbox não é portão: o encoding do dono quebrou o script" passa nos quatro.* Sem esse filtro, o arquivo de aprendizados enche de obviedade ao longo de 40 sessões — e aí ninguém lê o arquivo, **inclusive as linhas que importavam.**

NÃO USE PARA

No meio de uma implementação. E não conserta o que a lição apontou: lição vira card no backlog. Lição de um projeto só não vira regra do kit — regra do kit precisa de 2+ projetos.

PROVA SKILL.md:15 e :22 (erros do agente), :18 e :25 (frase desconfortável), :27-37 (filtro de quatro critérios), :43-45 (limites)

13. Agentes de arquitetura — 2 fichas

Decidem a forma antes de existir código. É aqui que vive o mecanismo mais contraintuitivo do kit: o agente cujo melhor resultado é dizer "não".

08 · arquitetura-monolito

PADRÃO

b_process/skills/architecture-monolith/

PARA QUE SERVE

Organiza o projeto como **uma coisa só para publicar**, mas com divisórias internas de verdade — de modo que amanhã dê para arrancar um pedaço e transformá-lo em serviço, se algum dia houver motivo medido.

GATILHO

"Como eu organizo as pastas deste projeto?". É o padrão: usa-se sempre que não houver decisão congelada de micros serviços.

O PORTÃO

Mapa de módulos × tabelas donas × dependências (acíclico) escrito no plano · **nenhum acesso a tabela de outro módulo — verificado por busca no código, não por confiança** · migrations rodam do zero e criam o banco inteiro · representações obrigatórias registradas antes do primeiro código · **gatilho de extração escrito.**

O QUE A TORNA BOA

Ela reduz "monólito modular" a **uma única regra falsificável**: *"Módulo A não faz SELECT na tabela de B — chama a porta de B. Esta é a única regra que decide se você tem um monólito modular ou uma bola de lama com pastas bonitas."* E o portão verifica isso por busca no código — a diferença entre uma regra e uma intenção. O segundo mecanismo é o **pré-compromisso**: o critério que autorizaria virar microserviço é escrito *hoje*, quando ninguém está emocionalmente investido. *"Vai escalar" não é gatilho; número medido é.*

NÃO USE PARA

Quando a decisão de microserviços já estiver tomada e congelada. Não escreve código de módulo, não decide o schema, não extrai serviço.

PROVA SKILL.md:14 ("vai escalar" não é gatilho), :19 (a regra única), :25 (representações no dia 1), :38-43 (portão), :49-51 (limites)

09 · arquitetura-microservicos

PORTÃO DE EXISTÊNCIA

b_process/skills/architecture-microservices/

PARA QUE SERVE

Decide se vale a pena quebrar o sistema em pedaços independentes — e, se valer, desenha quais são os pedaços, quem é dono de qual dado e em que ordem extrair. Na maioria das vezes a resposta certa é "ainda não".

GATILHO

Alguém propõe "vamos separar isso em serviços". Entra uma vez, na Fase 1.

O PORTÃO

Portão de existência com seis perguntas que só aceitam fato, não intenção — número de times reais hoje, a métrica que mostra a necessidade, a exigência escrita, se existe observabilidade distribuída, se existe automação de deploy. **Reprovado se:** um time só, ou sem observabilidade distribuída, ou sem automação de deploy. Se aprovar: grafo de dependências acíclico, cada aresta marcada síncrona ou assíncrona com justificativa, e o primeiro serviço extraído com deploy e **rollback** demonstrados.

O QUE A TORNA BOA

Três coisas. **A reprovação é automática e não negociável em três condições objetivas** — "um time só" é um número, não uma opinião. **Reprovar não deixa o dono de mãos vazias:** a mesma linha já prescreve a entrega alternativa, o que remove o incentivo de aprovar só para ter o que entregar. E o arquivo inverte o incentivo da IA de agradar, por escrito: *"A resposta correta mais frequente é 'ainda não' — e entregá-la é sucesso, não recusa."* Por fim, obriga a provar o ciclo completo no primeiro serviço **antes** de fatiar o segundo, impedindo a fatia em massa que produz 12 serviços e nenhuma operação.

NÃO USE PARA

Organizar módulos dentro de uma aplicação única. Não implementa serviço, não decide sozinha que sim (é D-NN do dono), não desenha o frontend.

PROVA SKILL.md:8 ("ainda não" é sucesso), :13-21 (as 6 perguntas), :23 (as 3 condições de reprovação + a alternativa), :33 (um serviço por vez), :38-44 (portão)

14. Agentes de backend — 3 fichas

A parte do sistema que, se sair errada, ninguém conserta depois — porque dado ruim sobrevive a qualquer reescrita de código.

10 · backend-dominio

b_process/skills/backend-domain/

PARA QUE SERVE

Escreve a regra de negócio e o jeito como o dado é guardado: schema, migration, endpoint, validação, transação, invariantes.

GATILHO

Qualquer tarefa que crie tabela, migration, endpoint, cálculo de domínio ou validação de servidor. Fase 2.

O PORTÃO

Migration roda num banco vazio e recria o schema inteiro · teste verde na máquina do dono cobrindo bordas (vazio, zero, ausente, limite, duplicado, divisão por zero) e **cada invariante · invariante testado também pela via mais baixa possível: tentar violar direto no banco falha** · operação transacional não deixa efeito parcial quando falha no meio.

O QUE A TORNA BOA

A regra de força do invariante: *constraint no banco > validação na aplicação > confiança no chamador*, porque "regra que só existe no código da aplicação é burlada pela próxima porta de entrada". E o portão verifica isso pela via mais baixa — o que **reprova a validação que existe só no controller**. Além disso, trava as representações obrigatórias antes da primeira linha (dinheiro em inteiro, data UTC, ID opaco), com o custo citado: **descobrir isso tarde causou 6 versões de schema em 7 dias num projeto real**. E proíbe consertar de carona: bug pré-existente vira QA-NN, não vira commit oportunista.

NÃO USE PARA

Agregar chamadas para uma tela (é o BFF) nem infraestrutura. Não desenha tela, não decide regra ambígua (registra Q-NN e para), não mexe em módulo que não é o da sessão.

PROVA SKILL.md:14-15 (representações + o custo), :19 (hierarquia do invariante), :25 (ausente ≠ zero), :30 (não conserta de carona), :35-41 (portão), :47-50 (limites)

11 • backend-bff

b_process/skills/backend-bff/

PARA QUE SERVE

Constrói a camada fina que junta dados de várias fontes no formato exato que uma tela precisa — e que protege o cliente das falhas e dos segredos dessas fontes.

GATILHO

Uma tela precisa de dados de mais de uma fonte, ou o payload da API não é o que a tela consome.

O PORTÃO

Amostra real de cada upstream na mão antes de começar — sem amostra, pare e peça · teste cobrindo caminho feliz, **upstream fora**, upstream lento (timeout), payload sem campo opcional e lista vazia · erro de upstream nunca vaza cru · falha parcial sinalizada no payload · escrita repetida não duplica efeito.

O QUE A TORNA BOA

Falha parcial vira contrato, não `try/catch`: devolva o que conseguiu e sinalize o que faltou; *nunca troque dado ausente por zero, vazio ou valor inventado*. O arquivo nomeia o custo com precisão cirúrgica: "catch que devolve `{}` e faz a tela mostrar zero como se fosse fato" — o defeito que passa em todos os testes e faz o dono decidir com base num número falso. E a skill carrega o próprio critério de escopo errado: "Se você está escrevendo regra de negócio aqui, você está no lugar errado — pare e devolva ao domínio."

NÃO USE PARA

Regra de negócio de domínio nem CRUD direto de uma tabela. Não cria tabela nem migration; não decide contrato de outro serviço.

PROVA SKILL.md:8 e :42 (autodetecção de escopo), :14 (amostra real), :21 (falha parcial), :22 (timeout com orçamento), :31-36 (portão), :51 (a armadilha do catch)

12 • microservice-sync

b_process/skills/microservice-sync/

PARA QUE SERVE

Conecta dois serviços numa chamada que espera resposta — e torna esse acoplamento explícito, limitado e testado, ou prova que a chamada não deveria ser síncrona.

GATILHO

Um serviço chama outro. Só entra se houver mais de um serviço.

O PORTÃO

STEP 0 que pode reprovar a própria forma: "pode ser feito depois, com o usuário seguindo em frente? ⇒ assíncrono (evento/fila) — registre D-NN e pare aqui" · chamada em cadeia: some os timeouts e verifique se o pior caso cabe no orçamento do usuário. Na entrega: teste com o serviço chamado **fora do ar** · teste com resposta lenta, com o timeout disparando no valor configurado · retry demonstrado como seguro (não duplica efeito) · pior caso de latência calculado.

O QUE A TORNA BOA

Ela **bloqueia antecipadamente a saída fácil**, que é literalmente a primeira coisa que qualquer pessoa faz: *"Se o pior caso não cabe, a resposta é mudar o desenho, não aumentar o timeout."* Ordena retry e idempotência na sequência correta — *chave de idempotência obrigatória antes de qualquer retry*, porque retry sem idempotência é a fábrica de cobrança dobrada. E os testes do portão são **testes de falha**, não de caminho feliz: a entrega só é aprovada por *como ela quebra*. A conta que justifica o orçamento de tempo distribuído: *cadeia $A \rightarrow B \rightarrow C$ onde cada um usa 30 s = o usuário espera 90 s por um erro*.

NÃO USE PARA

Fila ou evento assíncrono, nem chamada de UI para API. Não cria chamada síncrona sem timeout, retry e limite; não inventa o contrato do outro serviço.

PROVA SKILL.md:13–21 (STEP 0), :21 (não aumente o timeout), :25 (deadline propagado), :26 (idempotência antes do retry), :27 (retry imediato é DoS contra si mesmo), :36–41 (portão), :57 (a conta dos 90 s)

15. Agentes de frontend — 2 fichas

Um constrói a tela que uma pessoa real usa; o outro quase sempre diz que não se deve dividir o frontend.

13 · frontend-uiux

b_process/skills/frontend-uiux/

PARA QUE SERVE

Constrói a tela que uma pessoa real usa, com os estados que normalmente ficam para depois — carregando, vazio, erro — tratados como parte da entrega, não como polimento.

GATILHO

Qualquer tela, componente ou formulário; especialmente quando o usuário final for pessoa não técnica.

O PORTÃO

Build e typecheck verdes na máquina do dono · fluxo crítico executado ponta a ponta no **viewport mínimo e no desktop, com evidência** (o que você clicou e o que apareceu) · **os 4 estados demonstrados por tela nova** · nenhum texto técnico vazando para o usuário; nenhuma ação destrutiva sem confirmação · teclado percorre o fluxo crítico, com foco visível.

O QUE A TORNA BOA

Os **4 estados são condição de existência da tela**, não polimento: *"carregando · vazio · erro · sucesso. Falta um $\Rightarrow$ a tela não está pronta."* Isso reclassifica o que todo mundo trata como "depois eu faço" em requisito de aceite. E o portão de UX é um número, não uma sensação: *"escreva em quantas ações o fluxo crítico fecha (ex.: venda fecha em ≤ 3 ações) — ele é o portão de UX, não 'parece fluido'."* A armadilha paga mais específica do kit também mora aqui: *proteger com senha ou PIN uma tela do fluxo principal — o dono remove depois porque travava o caixa*.

NÃO USE PARA

Lógica de servidor, contratos de API ou infraestrutura. Não decide regra de negócio; não cria endpoint; **não entrega tela sem estado de erro e vazio**.

PROVA SKILL.md:15 (o número do fluxo crítico), :20 (os 4 estados), :25 (formatação só na exibição), :27 (não muda a API para facilitar a tela), :31–35 (portão), :49 (a armadilha do PIN)

14 · frontend-mfe

PORTÃO DE EXISTÊNCIA

b_process/skills/frontend-mfe/

PARA QUE SERVE

Decide se o frontend deve ser dividido em pedaços publicáveis separadamente — e quase sempre a resposta é não, porque micro frontend resolve um problema de *organização de times*, não um problema de código grande.

GATILHO

Alguém propõe dividir o front em *remotes* / Module Federation / "cada time cuida da sua parte".

O PORTÃO

Portão de existência: quantos times publicam frontend hoje, independentemente? **Menos de 2 ⇒ reprovado** — registre D-NN rejeitado e entregue uma SPA modular. Aprovado só com pelo menos um "sim forte", e **escrevendo no D-NN o custo que se está assumindo**. Na entrega: derrube um remote e mostre que o resto continua utilizável · bundle final inspecionado, sem dependência crítica duplicada em versões diferentes · deploy independente demonstrado.

O QUE A TORNA BOA

O portão **reprova por um número de uma cifra** — "menos de 2 times" não admite interpretação — e a mesma linha já antecipa a objeção emocional: *"Isso não é preguiça, é o resultado correto."* Aprovar obriga a escrever a conta (build mais complexo, versionamento entre remotes, duplicação de dependências, debug distribuído): quem aprova assina o custo. E ela verifica o **artefato**, não a intenção: *"duas versões do framework no mesmo bundle é o defeito clássico — verifique o bundle final."* A independência também é falsificável: *se só builda dentro do monorepo inteiro, a independência é fictícia.*

NÃO USE PARA

SPA única sem divisão. Não implementa antes do portão de existência — *reprovar é resultado válido e frequente.*

PROVA SKILL.md:13–20 (portão de existência), :15 ("menos de 2 ⇒ reprovado"), :20 (custo assumido no D-NN), :26 e :36 (bundle final), :30 (independência falsificável), :33–38 (portão de entrega)

16. Agentes transversais — 6 fichas

Entram em qualquer fase, quando o assunto aparece. Dois deles — testes e revisão adversarial — são o portão da Fase 4 e voltam com detalhe na Parte V.

15 · autenticação

b_process/skills/authentication/

PARA QUE SERVE

Define e implementa quem entra no sistema e o que cada um pode fazer — **sem trancar o que não precisa ser trancado**.

GATILHO

Assim que existir área sensível: login, PIN, rota protegida, "só o gerente pode ver isso".

O PORTÃO

Matriz *área × exigência* escrita no D-NN antes de qualquer código · para **cada** área: sem sessão, a página redireciona e a API retorna 401/403, com teste automatizado por rota sensível · **área declarada aberta continua acessível sem fricção** · o boot recusa iniciar com **segredo de exemplo**, e o segredo real não está no repositório · expiração e logout verificados · nenhuma rota nova aberta por omissão.

O QUE A TORNA BOA

É o **único arquivo do kit que reprova por excesso de segurança**: o portão verifica que a área declarada aberta *continuou* aberta, porque *proteção que atrapalha o fluxo principal é removida pelo dono depois* — e você paga duas vezes, uma para implementar e outra para remover. O segundo mecanismo: o **boot recusa iniciar com placeholder**, transformando um erro humano previsível numa falha barulhenta no minuto zero em vez de um vazamento silencioso. O arquivo declara que segredo de sessão fixo e boot com placeholder foram **os dois achados mais graves de um projeto real deste kit**. Terceiro: protege a API, não só a tela — *"o padrão de erro é proteger a tela e deixar o endpoint aberto"*. Quarto: nega por padrão, porque o inverso vaza.

NÃO USE PARA

Criptografia de dados em repouso nem chave de API entre serviços. Não decide *o que* proteger — isso é do dono.

PROVA SKILL.md:8 (pergunte o que proteger), :20 (a matriz), :24 (boot recusa placeholder), :26 (API, não só tela), :27 (nega por padrão), :36–42 (portão), :56 (os dois achados mais graves)

PARA QUE SERVE

Empacota a aplicação e descreve a infraestrutura em código, de modo que a máquina do usuário final receba algo pronto e que toda mudança de infra possa ser lida antes de ser aplicada.

GATILHO

Docker, compose, Terraform, "vamos subir isso". Fase 5.

O PORTÃO

`docker compose up -d` sobe num ambiente limpo, com healthcheck verde · **derrubar e subir de novo preserva os dados (comprovado, não presumido)** · se houver atualização automática: **rollback testado na máquina real** · `terraform plan` limpo e revisado, com todo *destroy/replace* explicado e aprovado pelo dono.

O QUE A TORNA BOA

Ela **separa quem propõe de quem aplica**, por regra: *"plan é o portão; apply é do dono. Nenhum apply em produção pelo agente."* É o único ponto do kit onde a ação irreversível fica fora do alcance do agente por *desenho*, não por permissão de ferramenta. E ela sabe onde olhar num plano que parece verde: *recurso com dado — banco, volume, bucket — sendo substituído é interrupção, e às vezes perda*. Duas outras: o segredo aparece no *state* do Terraform (*"proteja o state como se fosse o segredo"*) — o vazamento que quase todo tutorial omite; e a ordem importa — **automatizar a subida sem provar a descida é criar um sistema que só sabe piorar**.

NÃO USE PARA

Código de aplicação nem pipeline de teste. Não faz a máquina do usuário final compilar; não põe segredo em imagem, variável versionada ou log.

PROVA SKILL.md:21 (segredo em camada intermediária), :23 e :47 (versão em runtime), :25 (build no cliente é anti-padrão), :34 (plan × apply), :35 (destroy/replace), :36 (o state), :45-52 (portão), :50 e :71 (rollback antes da automação)

17 · testes

PARA QUE SERVE

Produz a evidência de que o comportamento é o esperado. No vocabulário do kit: **sem teste, a entrega não existe**.

GATILHO

Escrever, organizar ou consertar testes; ou qualquer pedido de prova de que algo funciona. Fase 4a.

O PORTÃO

Suíte verde **na máquina do dono**, com o comando declarado · cada regra nova com teste de caminho feliz e de borda · cada invariante do domínio com teste próprio · fluxo crítico coberto ponta a ponta, **sem dublê no caminho principal** · cada **QA-NN corrigido com teste de regressão que falha na versão antiga** · a suíte roda duas vezes seguidas com o mesmo resultado e **em ordem alterada** · nenhum teste depende de rede, horário ou estado deixado por outro teste.

O QUE A TORNA BOA

Ela **define o teste inútil e autoriza apagá-lo**: *"Cobertura é diagnóstico, não meta. Pergunte: que bug este teste pegaria? Se não houver resposta, apague."* Autorizar a remoção de teste é raro, e é o que impede a suíte de virar teatro de métrica. Segunda: *"comece pelo teste que falha — teste escrito depois, que já passa de primeira, frequentemente não testa nada."* Terceira: a lista de bordas é **fechada e enumerada** (vazio, zero, negativo, ausente, desconhecido, divisão por zero, limite exato e limite ± 1 , duplicado, maior e menor valor plausível) — uma lista fechada é auditável; "teste as bordas" não é. Quarta: *flaky é bug com prazo* — conserte ou remova, nunca "roda de novo até passar", porque suíte em que ninguém confia é pior que suíte inexistente.

NÃO USE PARA

Caçar bugs em código existente — isso é a revisão adversarial. Não altera o código sob teste para o teste passar.

PROVA SKILL.md:8 e :22 (comece pelo teste que falha), :23 (a lista de bordas), :27 (dublê de amostra real), :30 (apague o teste inútil), :31 (flaky), :33 (máquina do dono), :36-42 (portão), :48-50 (limites)

18 · guardrails-review

b_process/skills/guardrails-review/

PARA QUE SERVE

Uma sessão separada com um objetivo só: **quebrar o que foi construído**. Não melhora, não refatora, não elogia, não conserta — encontra, prova e reporta.

GATILHO

Antes de entregar; ao fechar um módulo ou milestone. Fase 4b, sempre em sessão separada de quem construiu.

O PORTÃO

Relatório registrado em `e_qa/<n>_qa_pass<NN>_report_<AAMDD>_<HHMM>.md` — sem relatório, a fase não aconteceu, mesmo com placar zero · as 12 frentes percorridas explicitamente, dizendo o que não deu para verificar e por quê · cada achado com reprodução executável e severidade · **você não consertou nada** · comparação com a passagem anterior.

O QUE A TORNA BOA

É a única das 24 que **define a condição em que ela deixa de ser a resposta**: três passagens sem queda no placar ⇒ declare o laço esgotado e recomende consistência de artefatos ou planejamento. Além disso: *"placar zero sem tentativa séria não é aprovação, é omissão"* — inverte a leitura natural de "não achei nada". E fecha a rota de fuga clássica de qualquer revisão sob pressão de prazo: *"restrição inegociável é crítico automático; a saída é ajustar o código, nunca reinterpretar a restrição até ela caber no que já foi feito."* Por fim, desmonta a falsa segurança: aprovar porque "os testes passam" não vale, já que — nas palavras do arquivo — *"a suíte pode não cobrir o defeito, ou validar um build antigo."*

NÃO USE PARA

Escrever feature nem consertar o que você encontrou sem autorização. Não refatora, não elogia, não inventa requisito.

PROVA SKILL.md:8 (placar zero é suspeito), :18 (amostra real de log), :25 (doc × código é achado), :34-38 (restrição inegociável), :49-55 (portão), :55 (detector de laço), :61-64 (limites), :74 (suíte verde não é aprovação)

19 · dependencias-supply-chain

b_process/skills/dependencies-supply-chain/

PARA QUE SERVE

Decide o que entra no projeto vindo de fora — e a maior parte do trabalho é **não deixar entrar**. Toda biblioteca é código de estranho rodando com os seus privilégios.

GATILHO

"Vou instalar a lib X", "precisa subir essa versão", "apareceu uma CVE", conflito de lockfile.

O PORTÃO

STEP 0 com uma tabela cuja coluna se chama literalmente **"Reprova se"**: a plataforma já faz isso? · quanto código eu escreveria (menos de ~50 linhas triviais: escreva) · quantas dependências transitivas ela traz — *veja a árvore antes, não depois* · está viva? · a licença é compatível? Na entrega: **uma atualização por vez, com a suíte verde registrada entre cada uma** · nenhuma dependência de desenvolvimento na imagem de produção, **conferido no artefato, não no manifesto**.

O QUE A TORNA BOA

A primeira pergunta do STEP 0 elimina a maior categoria de dependência inútil antes de qualquer análise técnica: *a plataforma já resolve isso?* E ela **fecha a saída de escape da CVE** enumerando as saídas legítimas: *"atualizar · mitigar (a rota vulnerável não é exercitada, e você prova isso) · aceitar com D-NN e prazo. A quarta — ignorar em silêncio — não existe."* Enumerar as saídas válidas e declarar a quarta inexistente é muito mais forte que "trate as CVEs". Duas menores mas raras: **remoção é entrega** (a dependência órfã de uma decisão revertida continua sendo superfície de

ataque) e **plano B** escrito para dependência crítica, porque descobrir que não há plano no dia do abandono é o pior momento possível.

NÃO USE PARA

Código próprio da aplicação nem infraestrutura. Não adiciona dependência sem alternativa avaliada — *inclusive a de não adicionar*. Não sobe versão major junto com feature.

PROVA SKILL.md:16–22 (a tabela "Reprova se"), :29 (uma por vez), :31 (as três saídas da CVE), :32 (severidade é do seu contexto), :36 (remoção é entrega), :38 (plano B), :41–50 (portão)

20 • privacidade-dados-pessoais

b_process/skills/privacy-personal-data/

PARA QUE SERVE

Garante que o sistema só guarda dado de pessoas que precisa guardar, pelo tempo que precisa, e sabe devolvê-lo ou apagá-lo quando pedirem.

GATILHO

O sistema guarda cadastro de pessoas. **Entra junto com o schema, não depois** — coluna criada sem finalidade vira obrigação permanente, e retenção retroativa é migração dolorosa.

O PORTÃO

STEP 0 de inventário obrigatório — uma linha por campo pessoal, com finalidade concreta; *"pode ser útil depois" não é finalidade, é motivo para não coletar* · varredura de log por dado pessoal, conferida numa **amostra real** · exclusão a pedido do titular testada ponta a ponta, com a lista do que **não** é apagado e por quê · rotina de retenção implementada, com teste · cifrado em repouso e em trânsito, **inclusive backup**.

O QUE A TORNA BOA

A **entrega mais valiosa dela é a remoção de campo**: *"campo sem finalidade concreta é candidato a não existir — proponha a remoção"*, porque *dado que você não coleta não vaza, não expira, não precisa ser exportado e não precisa ser apagado*. Ela também **recusa explicitamente o papel jurídico**: base legal e prazo de retenção são decisões do dono, cada uma vira Q-NN — o que impede o erro caro de a IA inventar conformidade. E resolve, no schema e não na crise, a colisão entre "nunca exclua histórico" e o direito de exclusão: *anonimizar o titular preservando o fato — a venda continua existindo, o comprador vira registro anônimo*. Duas frases que fecham a seção: *"prazo escrito em documento e nenhuma rotina que apaga é prazo que não existe"* — e aí o documento vira prova contra o projeto; e *"se você não consegue exportar, é porque não sabe onde o dado está — e isso é o achado."*

NÃO USE PARA

Controle de acesso (é autenticação) nem segurança de infraestrutura. Não coleta "por precaução"; não decide base legal sozinha.

PROVA SKILL.md:8 (dado não coletado não vaza), :10 (não é advogado), :15–27 (inventário), :27 (a remoção como entrega), :33 (teste não recebe dado de produção), :34 (anonimizar ≠ pseudonimizar), :36 (a conciliação no schema), :38 e :79 (retenção implementada), :44–53 (portão)

17. Agentes de sistema vivo — 3 fichas

Existem porque um projeto passa muito mais tempo sendo mantido do que sendo criado — e a versão 4 do kit só tinha agente para a criação.

21 • depuracao-diagnostico

b_process/skills/debugging-diagnosis/

PARA QUE SERVE

Descobre a **causa** de um problema que já apareceu, **prova** que é aquela causa, e só então conserta — com um teste que impede o bug de voltar.

GATILHO

"Quebrou", "deu erro", "está errado". E também "está lento **desde ontem**" — porque algo mudou, e achar o quê é mais barato que otimizar.

O PORTÃO

STEP 0 com três perguntas bloqueantes — *"se as três não estiverem respondidas, pare e pergunte: investigar sem elas é como escrever parser sem amostra real"* · reprodução determinística escrita, com observado × esperado · **causa-raiz provada por liga/desliga, não inferida por proximidade temporal** · hipóteses descartadas listadas, cada uma com o fato que a matou · teste de regressão citando QA-NN, **que falha na versão anterior (demonstrado)**.

O QUE A TORNA BOA

Ela trata a **não-reprodução como entregável, não como fracasso**: *"não conseguiu reproduzir? Isso é a entrega desta sessão — diga o que falta e pare. Não conserte o que você não viu quebrar."* Isso remove a pressão que faz a IA "consertar" no escuro. Ela também **define o que conta como prova**: *"eu ligo a causa e o defeito aparece, eu desligo e some. Correlação com o momento do deploy não é prova."* E faz a pergunta que ninguém faz: *"o dono está olhando o que ele acha que está olhando?"* — cache de dev server, build antigo, container com imagem anterior, CDN, service worker, banco de outro ambiente. O custo de pular essa pergunta está escrito: *você conclui que a correção não funcionou e reescreve algo que já estava certo.*

NÃO USE PARA

Caçar bugs que ninguém relatou (é a revisão adversarial) nem escrever feature nova. Não conserta antes de reproduzir — *correção sem reprodução é chute com commit*. Não silencia exceção para o sintoma sumir.

PROVA SKILL.md:15-22 (STEP 0), :18 (código ou falta de dado), :20 (o dono está olhando o quê), :25 (não-reprodução é entrega), :28 (o que é prova), :32 (não silencie), :34 (não conserte de carona), :39-46 (portão)

22 · performance

b_process/skills/performance/

PARA QUE SERVE

Troca lentidão medida por rapidez medida — e recusa o resto. A maior parte do trabalho é descobrir que o gargalo não está onde todo mundo achava.

GATILHO

"Está lento" *sem* baseline. Com "desde ontem", o caso é de depuração.

O PORTÃO

Sem estes quatro números, não comece: quanto está demorando (medido) · quanto *deveria* demorar · com que volume de dado · é reproduzível (3 medições no mesmo cenário; variação de 10× significa que você está medindo ruído). **Reprovado se ninguém sabe dizer quanto deveria demorar**. Na entrega: gargalo apontado por **profiler ou plano de execução, com a evidência anexada** — não por leitura de código · novo baseline registrado.

O QUE A TORNA BOA

Ela **reprova por ausência de alvo**, e explica: *otimizar sem alvo não termina nunca e sempre piora a legibilidade*. Proíbe a intuição por escrito, com o motivo: *"a intuição de quem escreveu o código erra sistematicamente — é justamente onde ele não olhou."* Impede a troca de um bug barato por um caro: *"cache sem regra de expiração é dado velho garantido — você acabou de trocar um bug de lentidão por um bug de correção, que é pior"*, e o segundo demora meses para ser notado. E ela **deixa o instrumento para a próxima sessão**, não só o ganho: o novo baseline é contra o que a próxima regressão será detectada. Por fim, admite que a resposta certa pode ser não mexer: *"medi e está dentro do orçamento; otimizar aqui é cruft — entregue isso como D-NN rejeitado; é sucesso, não recusa."*

NÃO USE PARA

Bug de correção; nem para otimizar algo que ninguém mediu como problema. Não muda duas coisas ao mesmo tempo — o ganho fica sem causa.

PROVA SKILL.md:10 (a recusa é sucesso), :15-23 (os quatro números), :23 (reprovado sem alvo), :27 (contra a intuição), :31 (cache sem expiração), :32 (correção > velocidade), :37 e :48 (novo baseline), :40-48 (portão), :61 (reportar o que não deu ganho)

PARA QUE SERVE

Faz o sistema contar o que está fazendo enquanto ninguém olha, para que, quando algo quebrar, o dono descubra o que houve **sem precisar abrir o código**.

GATILHO

O sistema passa a rodar continuamente; ou a frase "não sei o que aconteceu em produção". Precisar depurar duas vezes o mesmo sintoma é o sinal de que faltou isto.

O PORTÃO

As 3 perguntas do STEP 0 são respondíveis **sem abrir o código — demonstrado, não presumido** · varredura por dado pessoal e segredo nos logs gerados, feita **numa amostra real de log** · endpoint de saúde que verifica dependência essencial e informa a versão em execução · **cada alerta com limiar justificado, dono e ação escrita no RUNBOOK** · nenhuma regressão de latência no fluxo crítico por causa da instrumentação.

O QUE A TORNA BOA

Um **critério de aceite único, concreto e datado**: "quando algo quebrar às 14h de uma terça, o dono descobre o que houve sem abrir o código? Se a resposta é não, você não entregou observabilidade — entregou arquivos de log." É a diferença entre instalar uma ferramenta e resolver o problema. E, ao contrário de praticamente todo guia do assunto, esta skill **manda NÃO instrumentar**: "log que ninguém lê é custo de disco, ruído no diagnóstico e risco de vazamento — se ninguém lê, não registre." Ela também corrige o erro clássico de alerta: "fluxo de venda com 20% de erro" é acionável; "uso de memória em 80%" é normal em metade dos sistemas e gera alerta que ninguém mais lê." E duas armadilhas com nome: o **healthcheck que fica verde com o banco fora**, e a **rotação de log ausente** — o disco enche e derruba o sistema, causado pela ferramenta que existia para evitar quedas.

NÃO USE PARA

Trace entre serviços distribuídos nem depurar um bug específico. Não cria alerta sem dono e sem ação; não instrumenta o que ninguém vai olhar.

PROVA SKILL.md:8 (as 14h de uma terça), :18 e :23 (não instrumente tudo), :33 (alerte no sintoma), :34 (alerta com dono), :35 e :73 (healthcheck falso), :36 e :74 (rotação), :40-48 (portão), :42 e :54 (amostra real de log)

18. Agente de dados e análise — 1 ficha

O único cujo entregável é **um número**, não uma tela. Aí o portão deixa de ser "o teste passa" e vira "a amostra sustenta a afirmação".

24 · dados-analise

PARA QUE SERVE

Produz números em que se pode confiar — e, mais importante, **destrói a confiança nos números que não a merecem**. Coleta, parser/ETL, features, modelo, métrica, e qualquer afirmação do tipo "melhorou 3%".

GATILHO

Fonte de dados externa, parser, dataset, backtest, métrica. Entra **antes** do backend de domínio quando o projeto nasce de uma fonte externa — é ela que traz a amostra real sem a qual o schema é chute.

O PORTÃO

Reprovado se ninguém sabe que decisão o número muda: construir é cruft — entregue como D-NN rejeitado. Na entrega: parser escrito a partir de **amostra real anexada** · coleta interrompida no meio preserva o já coletado e retoma (**testado de verdade, matando o processo**) · afirmação numérica com n, intervalo de confiança e seed declarados, comparada contra baseline nomeado · **duas execuções, mesmo resultado**.

O QUE A TORNA BOA

Ela não só proíbe o parser sem amostra — **define o entregável substituto**: "se não tem amostra, o primeiro entregável é o modo `--investigar` que a busca, não o parser." Nomeia o defeito que atravessa a suíte inteira: **ausente virando zero**

"passa em todos os testes, envenena a métrica e só aparece quando alguém questiona um número que já foi usado para decidir". Exige rebuild ao mudar fórmula, porque derivado antigo convivendo com fórmula nova é a origem de todo número conflitante. E mata a fraude estatística mais convincente que existe, a generalização indevida: "o que passou o portão num dataset, período ou liga re-passa no novo. O dado mudou; o portão vale de novo." Fecha com o vazamento treino/teste: embaralhar série temporal fabrica ganho que não existe — e o ganho fabricado é convincente o bastante para ninguém desconfiar.

NÃO USE PARA

Regra de negócio transacional nem para a tela que mostra o resultado. Não afirma número sem incerteza declarada; não decide o que o número significa para o negócio.

PROVA SKILL.md:21 (reprovado sem decisão), :24 (o modo --investigar), :26 e :61 (ausente vira zero), :29 (split temporal), :33 (rebuild ao mudar fórmula), :34 (ganho não transfere), :38-46 (portão), :52-54 (limites)

ACHADO EDITORIAL ENCONTRADO AO PREPARAR ESTE DOSSIÊ

O arquivo `data-analysis/SKILL.md` tem duas seções com o mesmo título "Armadilhas pagas" (linhas 59 e 67), com conteúdo parcialmente sobreposto. Nenhuma das outras 23 skills repete um cabeçalho. Vale consolidar — está registrado aqui em vez de ser corrigido em silêncio, porque é exatamente a espécie de divergência que o próprio kit classifica como achado.

19. Por que um kit de processo tem testes

Esta é a parte que costuma surpreender: um repositório de documentação com 57 testes automatizados rodando em duas plataformas a cada push. A razão é simples, e vale dizê-la antes de mostrar qualquer código.

Um kit de processo promete que certas coisas **não vão passar**: contexto estourado, estado divergente, segredo versionado, ID citado que não existe. Essa promessa é verificada por um script — o `check.py`. Mas então surge a pergunta que quase ninguém faz: **quem verifica o verificador?**

A resposta importa porque o modo de falha é silencioso. Uma checagem que para de checar não grita: ela continua imprimindo verde. Foi exatamente o que aconteceu, e está medido — a Parte VI mostra o número. Por isso a arquitetura de testes do kit tem três camadas:

AS TRÊS CAMADAS DE VERIFICAÇÃO

Camada 1 · <code>check.py</code>	verifica o PROJETO → 14 checagens que reprovam o commit, 16 que avisam
Camada 2 · <code>test_check.py</code>	verifica o CHECK.PY → 57 testes, incluindo uma "isca" por checagem numerada
Camada 3 · <code>.github/workflows</code>	verifica que as camadas 1 e 2 rodam FORA do sandbox → Linux e Windows, a cada push, com histórico completo

A FRASE QUE RESUME A PARTE V

Uma checagem que não reprova o próprio caso não é portão — é decoração que imprime verde. O kit trata isso como uma regra testável: **toda checagem numerada precisa ter uma isca que prove que ela morde**, e existe um teste que reprova o commit se alguém adicionar uma checagem sem isca.

PROVA `scripts/test_check.py:750-758` — o teste `test_toda_falha_numerada_tem_isca` lê o cabeçalho do `check.py`, extrai os IDs numerados e exige que o conjunto de iscas seja idêntico.

O que foi rodado para este dossiê

A suíte inteira foi executada durante a preparação deste documento, para que nenhum número aqui dependa de o repositório dizer que passa.

```
$ python3 scripts/test_check.py
Ran 57 tests in 31.584s

OK

$ python3 -m pytest scripts/test_check.py -q
57 passed, 24 subtests passed in 31.96s
```

O runner oficial é o `unittest` da biblioteca padrão, não o `pytest` — o kit não tem dependência externa, e é isso que o faz rodar em qualquer máquina com Python, sem `pip install` e sem rede. O `pytest` foi usado apenas como conferência cruzada. Os 24 *subtests* vêm de quatro laços parametrizados: 3 separadores, 3 grafias, 4 formas de declarar o limite de WIP e as 14 iscas.

```
PROVA scripts/test_check.py:4 - "Só biblioteca padrão (unittest): o kit não tem dependências" ·
test_check.py:1051 - unittest.main(verbosity=2) .
```

20. `check.py` — o portão automático, checagem por checagem

718 linhas, zero dependências. Roda em todo commit se o hook estiver instalado, e é o que separa "o processo é uma boa ideia" de "o processo é cobrado".

O princípio de desenho está escrito no topo do arquivo, e é honesto: *"Cada checagem existe porque uma regra do kit era só prosa e alguém a pulou. As regras que a máquina consegue julgar, ela julga aqui; o que sobra é honestamente do dono."*

As 14 checagens que reprovam o commit

#	CHECAGEM	O QUE ELA DETECTA, EM LINGUAGEM SIMPLES	LINHA
1	Orçamento do contexto-fonte	O arquivo que toda sessão carrega passou de 4.000 caracteres — ou sumiu	197
2	Registro de decisões inchado	O arquivo de decisões passou de 12.000 caracteres	216
3	Fonte única	O mesmo nome de arquivo (backlog, contexto, decisões) aparece em dois lugares — estado duplicado	236
4	WIP acima do declarado	Mais tarefas "em andamento" do que o limite escrito no cabeçalho do backlog	243
5	Cruft óbvio	Arquivos <code>.bak</code> , <code>.tmp</code> , <code>.orig</code> versionados	264
6	Skill sem <code>name / description</code>	Uma skill sem o bloco que a torna instalável e disparável	269
7	Wikilink sem destino	Um link <code>[[assim]]</code> que não resolve para nenhuma nota	304
8	Segredo versionado	Segredo na árvore e no histórico do git — 30 commits por padrão, tudo com <code>--historico-completo</code>	349
9	<code>.gitignore</code> sem cobertura	Falta <code>.env</code> , <code>*.pem</code> , <code>*.key</code> , <code>id_rsa</code> ... em linha efetiva — comentário não conta	470
10	ID citado que não existe	Um <code>D-NN</code> , <code>Q-NN</code> ou <code>QA-NN</code> citado numa nota mas ausente do registro e do arquivo-morto	484
11	ID duplicado	O mesmo ID em duas linhas da tabela viva — o registro é append-only	500
12	"Em andamento" divergente	O backlog diz uma tarefa e o contexto diz outra	517
13	Tarefa → módulo inexistente	Uma tarefa aponta para <code>M42</code> e o plano não tem <code>M42</code>	518
14	Skill fora do esquema	Falta uma das seções obrigatórias: Contexto, Limites ou Saída	292

Os 16 avisos, que não reprovam

A distinção entre falha e aviso não é arbitrária. Uma delas tem história: a "nota órfã" (nota que ninguém linka) era falha e virou aviso, porque bloquear um commit por motivo cosmético ensina o usuário a rodar `--no-verify` — e aí ele deixa de rodar o portão inteiro. Reprovar demais desliga o portão.

#	AVISO	O QUE DETECTA	LINHA
1	frontmatter ausente	Nota sem o bloco de metadados no topo	567
2	placeholders	O contexto ainda tem <code><NOME DO PROJETO></code>	689

#	AVISO	O QUE DETECTA	LINHA
3	template em rascunho	Plano, decisões ou backlog ainda com <code>status: rascunho</code>	696
4	nota órfã	Nota que ninguém linka (era falha; virou aviso de propósito)	344
5	arquivo grande não varrido	Arquivo acima de 1 MB foi pulado pela varredura — o pulo é declarado, nunca silencioso	413
6	varredura de histórico não rodou	Timeout, erro de subprocesso ou ausência de repositório git	442
7	pre-commit não instalado	O hook não existe ou não chama o <code>check.py</code>	463
8	módulo do plano sem tarefa	Existe um módulo que ninguém vai construir	545
9	<code>description</code> sem fronteira negativa	Skill sem "Não use para... (é <i>outra skill</i>)" — sem isso duas skills disputam a mesma tarefa	299
10	contexto perto do teto	Acima de 3.600 de 4.000 — avisa antes da parede	211
11	decisões perto do teto	Acima de 9.600 de 12.000, já listando candidatas a arquivamento	231
12	tema fora do mapa de leitura	Documento de domínio que o contexto não cita — doc fora do mapa nunca é lido	578
13	sessão sem skill declarada	Entrada do changelog sem <code>**Skill:**</code> , ou com skill inexistente	604
14	ocupação declarada divergente	O contexto afirma "3200/4000" e o arquivo tem outro tamanho	628
15	questão do dono ausente do contexto	Q-NN aberta no registro e fora do único arquivo que toda sessão lê	645
16	achado grave vencido	QA crítico/alto aberto há mais de 14 dias — e, se a tabela não tiver a coluna, o script diz em voz alta que não julgou	675

UM DETALHE DE DESENHO QUE VALE A PENA MOSTRAR À BANCA

O aviso 16 tem duas saídas: uma quando encontra um achado vencido, e outra quando **não consegue julgar** porque a tabela não tem a coluna de data. Um script que emudece quando não sabe é indistinguível de um script que aprovou. Aqui, não saber é uma saída explícita. O mesmo princípio aparece no aviso 5 (arquivo grande pulado) e no aviso 6 (varredura de histórico que não rodou).

PROVA `scripts/check.py:681` · o mesmo princípio está no comentário do arquivo de aprendizados: *"Lição escrita não é lição aplicada: o comentário de uma checagem já nomeava 'checagem que emudece' enquanto outra, no mesmo arquivo, tinha a doença – conserte a espécie (um teste que cobre a classe), não o caso."* (`b_process/d_agent_learnings.md`)

21. `test_check.py` — 57 testes, classe por classe

1.051 linhas, 18 classes, 57 testes, 24 subtests. Cada classe protege uma espécie de defeito, e o docstring de cada uma conta qual bug a fez nascer.

Os testes montam um repositório git temporário a partir do kit inteiro, commitam, plantam um problema e verificam que o portão reprova. Não há *mock* no caminho principal: o teste roda o script de verdade e lê a saída de verdade.

#	CLASSE	TESTES O QUE ELA PROTEGE
1	<code>TestEncoding</code>	4 Caminho com acento e caractere fora do cp1252 derrubavam os scripts em Windows em português. Cobre a leitura e a escrita — ver seção 23.

#	CLASSE	TESTES O QUE ELA PROTEGE
2	<code>TestVarreduraDeHistorico</code>	3 Em <i>worktree</i> ou submódulo, o <code>.git</code> é um arquivo, não uma pasta — e a varredura de histórico sumia em silêncio, com saída verde.
3	<code>TestPortaoInstalado</code>	2 Cobra que o próprio pre-commit esteja instalado, e que um <code>core.hooksPath</code> customizado não gere aviso falso.
4	<code>TestProjetoNovo</code>	1 Projeto recém-criado tem de nascer passando no próprio portão .
5	<code>TestAtualizacao</code>	2 O <code>--upgrade</code> atualiza o <i>processo</i> sem encostar na <i>verdade</i> do projeto. O docstring o chama de "o teste mais importante do arquivo" — errar aqui apaga trabalho do dono.
6	<code>TestCustomizacaoPreservada</code>	2 A atualização não sobrescreve arquivo do kit que o dono editou; <code>--forçar</code> traz a versão do kit de volta.
7	<code>TestCoberturaModuloTarefa</code>	4 Cruza os módulos do plano com as tarefas do backlog: módulo sem tarefa avisa, tarefa apontando módulo inexistente reprova, template não preenchido não dispara falso positivo.
8	<code>TestEsquemaDasSkills</code>	3 As 24 skills seguiam o esquema Contexto/Limites/Saída por <i>hábito</i> ; aqui vira regra cobrada.
9	<code>TestHonestidadeDeclarada</code>	1 Reconta, a partir dos arquivos, a frase de auto-cobertura do README ("284 itens, o script julga 30, cerca de 11%"). Ela já mentiu uma vez.
10	<code>TestDocumentacaoNaoMente</code>	3 Contagem de agentes, scripts citados e índice de skills conferidos contra o disco. Nasceu porque quatro documentos ficaram dizendo "23 agentes" depois de entrar a 24ª.
11	<code>TestCanarioDosTemplates</code>	5 Injeta violação real nos templates como eles são entregues . Se alguém reformatar um cabeçalho e o parser parar de reconhecê-lo, a violação passaria — e este teste acusa.
12	<code>TestTodaChecagemTemIsca</code>	3 +14 O portão do portão . Uma isca canônica por checagem numerada, mais a meta-regra que exige isca para toda checagem nova, mais a contraprova. Detalhado na seção 22.
13	<code>TestInstrumentacaoDaSessao</code>	5 Cobra o campo <code>**Skill:**</code> nas entradas do changelog — sem ele, "qual agente paga o próprio custo" só se responde por arqueologia de git.
14	<code>TestNumeroDeclarado</code>	3 A ocupação declarada no contexto é conferida contra o tamanho real do arquivo; e um número alheio ao orçamento não pode disparar aviso falso.
15	<code>TestFilaDoDono</code>	3 Q-NN aberta no registro e ausente do contexto: fora dele, a pergunta não está sendo feita a ninguém.
16	<code>TestAchadoVencido</code>	4 Achado crítico/alto aberto há mais de 14 dias avisa; e quando a tabela não tem a coluna, o script diz que não julgou em vez de emudecer.
17	<code>TestArquivar</code>	6 O <code>arquivar.py</code> : decide o que pode sair do registro de decisões, relata por padrão e só escreve com <code>--aplicar</code> .
18	<code>TestIdArquivado</code>	3 ID retirado da tabela continua sendo ID real. Sem isso, uma correção anterior faria 22 IDs legitimamente arquivados virarem fantasmas — e o portão reprovaria todo commit.
Total		57 + 24 subtests

PROVA As linhas de início de cada classe: `test_check.py` :111 · :180 · :217 · :244 · :270 · :351 · :410 · :452 · :491 · :551 · :584 · :657 · :780 · :825 · :859 · :893 · :932 · :998

UM DETALHE METODOLÓGICO QUE A BANCA TÉCNICA VAI VALORIZAR

Duas classes declaram, no próprio docstring, a mesma regra: **testar o resultado, nunca o regex** — *"teste que espelha a implementação quebra junto com ela e não protege nada"*. É por isso que os testes plantam um problema e conferem a *saída* e o *código de retorno*, em vez de verificar se uma expressão regular está escrita de um jeito específico.

PROVA `test_check.py:594-595` e `:672-673`

Uma observação sobre `.pytest_cache/`

A pasta `.pytest_cache/v/cache/nodeids` existe no repositório e lista **23 testes em 8 classes**. Ela está **obsoleta**: o arquivo atual tem 57 testes em 18 classes. Quem olhar só esse cache subestima a suíte em cerca de 60% — e não veria a classe mais importante, a das iscas. Isso está aqui porque alguém da banca pode abrir a pasta; a fonte da verdade é rodar `python scripts/task.py test`.

22. As iscas: o portão que testa o portão

Se a apresentação puder mostrar só uma coisa técnica, que seja esta. É o mecanismo mais incomum do repositório, e ele nasceu de um erro real, medido, que sobreviveu a uma auditoria externa.

A história que fez a classe nascer

A checagem número 10 — "ID citado que não existe" — filtrava o texto com uma função que descarta trechos entre crases. E a casa escreve os IDs entre crases: `'D-13'`, não `D-13`. Resultado medido no primeiro projeto real: das 341 citações de ID, 300 estavam entre crases. A checagem enxergava 12% do que dizia enxergar e imprimia verde sobre o resto.

E já existia um canário para essa checagem. Ele passava. Porque o canário escrevia o ID **sem** crases — do jeito que ninguém escreve. A cegueira sobreviveu a uma auditoria externa *que citou o portão como ponto forte*.

A LIÇÃO, QUE VIROU REGRA DO KIT

Isca tem de sabotar como a casa escreve. Teste que passa pelo motivo errado é pior que teste ausente — porque ele produz confiança sem produzir cobertura.

PROVA `b_process/d_agent_learnings.md`, seção "Herdadas" · o número (12%, 341 citações, 300 entre crases) está em `docs/c_field_evaluation_tapgo_260813.md:45-49` · a correção é o QA-14, kit v13.2.

Como o mecanismo funciona

O CICLO DE UMA ISCA

- | | | |
|------------------------------|--|------------------------------------|
| 1. <code>montar_kit()</code> | copia o kit inteiro, roda git init, commita | (<code>test_check.py:92</code>) |
| 2. <code>iscas()</code> | planta UM problema específico e realista | (<code>test_check.py:693</code>) |
| 3. <code>rodar_check</code> | executa o portão de verdade, no repositório de verdade | (<code>:764</code>) |
| 4. <code>assert</code> | exige código de retorno 1 E o trecho de mensagem certo | |

A dupla asserção é deliberada. Só o código de retorno faria o teste passar por qualquer outro motivo — inclusive por um problema não relacionado no repositório. Exigir também o texto da mensagem prova que reprovou *pelo motivo pretendido*.

As 14 iscas, uma por checagem

CHEC.	O QUE A ISCA PLANTA	TRECHO DE MENSAGEM EXIGIDO
1	Anexa 4.100 caracteres ao contexto-fonte	orçamento: 4.000
2	Anexa 12.100 caracteres ao registro de decisões	acima de 12.000
3	Cria um segundo arquivo com o mesmo nome do backlog em outra pasta	duplicado
4	Troca 1 tarefa por 3 sob um cabeçalho que declara máximo 1	limite declarado
5	Cria um lixo.bak	Cruft
6	Skill com name: mas sem description:	sem 'description:'
7	Wikilink apontando para uma nota que não existe	sem destino
8	Escreve um segredo em formato realista num arquivo versionado	segredo versionado
9	.gitignore em que os padrões de segredo aparecem só em comentário, sem linha efetiva que os cubra	.gitignore sem cobertura
10	Cita `D-77` entre crases, de propósito — do jeito que a casa escreve	D-77
11	Repete uma linha de ID já existente na tabela	ID duplicado
12	Backlog diz T-42, contexto diz T-99	'Em andamento' divergente
13	Tarefa apontando para um módulo que o plano não tem	módulo inexistente
14	Skill com Contexto e Saída, mas sem a seção Limites	## Limites

PROVA test_check.py:692-748 — as 14 iscas, no método iscas().

As três cegueiras que a classe impede

TESTE	IMPEDE QUE...
test_toda_falha_numerada_tem_isca test_check.py:750	...alguém adicione uma checagem nova e nunca prove que ela pega alguma coisa . O teste lê o cabeçalho do check.py, extrai os IDs numerados e exige que o conjunto seja idêntico ao das iscas. Checagem sem isca reprova aqui .
test_cada_isca_reprova test_check.py:760	...uma checagem pare de checar e o verde continue saindo. É a cegueira do QA-14, agora coberta para toda a classe de defeitos, não só para aquele caso.
test_kit_entregue_passa_sem_isca test_check.py:771	...uma isca reprova por acidente , por causa de outro problema qualquer do repositório, e passe por prova válida. É a contraprova — e foi ela que denunciou, na preparação deste dossiê, que dois arquivos não tinham sido copiados corretamente.

Duas confissões escritas dentro do código

Duas iscas trazem, em comentário, o bug encontrado **ao escrever a própria isca**. Isso é raro o suficiente para ser mostrado à banca:

Isca 9: trocar só .env no .gitignore deixava .env.local, que contém a substring .env — então a isca não sabotava nada. Comentário no arquivo: "falso verde, achado ao escrever esta classe."

Isca 10: "escreve o ID ENTRE CRASES de propósito: é como a casa escreve, e era exatamente o caso que passava. Sem crases o teste não prova nada."

Há ainda um guarda de terceiro nível: a função que aplica a isca falha com a mensagem *"a âncora da isca sumiu do template — a isca deixou de sabotar"* se o texto que ela substitui desaparecer do template. **Isca que deixou de sabotar é isca morta, e ela grita.**

23. Windows, encoding e o CI em duas plataformas

Dois bugs que o kit já pagou **não reproduzem no Linux**. É por isso que o CI roda nas duas plataformas — e é o melhor exemplo concreto de "sandbox não é portão".

Os dois bugs, que são gêmeos

ID	ONDE	O QUE ACONTECIA
QA-01	Leitura	O git emite caminhos em UTF-8. Ler com o encoding do sistema — cp1252 num Windows em português — e um caminho com acento (<i>"Área de Trabalho"</i> , nome real da pasta sob OneDrive em português) derrubava a thread leitora. Pior: o erro é da família <code>ValueError</code> , então os tratamentos de exceção existentes passavam ao largo e o dono via um erro genérico a duas funções da causa.
QA-05	Escrita	Saída redirecionada num Windows em português usa cp1252. Um caractere fora dele — uma seta, um "≤" — matava o script na hora de imprimir, depois de todo o trabalho já ter sido feito.

O RESULTADO MEDIDO DO QA-01, ESCRITO NO COMENTÁRIO DO CÓDIGO

"O portão nunca rodou na máquina do dono, e todo 'OK' veio do sandbox Linux do agente."

PROVA scripts/check.py:44-51 — o comentário que documenta a causa, o efeito e a correção (a frase citada está em :49-50). A rede de segurança da saída, gêmea dela, está em :56-63.

Os quatro testes que cobrem isso

TESTE	O QUE FAZ
test_toda_chamada_ao_git_fixa_o_encoding :116	Analisa o código-fonte dos scripts e exige que toda chamada ao git fixe o encoding explicitamente. Roda em qualquer sistema operacional — é uma checagem estática do próprio código.
test_scripts_rodam_em_caminho_acentuado :129	Monta o kit dentro de uma pasta chamada <i>"Área de Trabalho"</i> e roda os scripts exigindo zero erro. <i>"No Windows este teste exercita cp1252 de verdade. No Linux ele só garante que nada mais quebrou — e é por isso que o CI roda nos dois."</i>
test_saida_nao_morre_em_console_cp1252 :139	Força o encoding de saída para cp1252 e roda o criador de projeto, exigindo que não haja erro de codificação. Reproduzível em qualquer sistema , o que o torna útil também no Linux.
test_nenhum_print_usa_caractere_fora_do_cp1252 :155	Varre todo texto literal dentro de chamadas de impressão em quatro scripts e falha se algum caractere não couber em cp1252. Distinção fina: o teste anterior guarda a <i>rede</i> de segurança; este guarda o <i>texto</i> — com a rede ligada, a seta não mata, mas sai como "?".

O CI, e por que ele existe

O comentário no topo do arquivo de workflow é a justificativa mais direta do kit inteiro:

"Até aqui o portão do kit nunca tinha rodado fora do sandbox de quem estava editando. Uma auditoria concluiu 'passa depois da correção, quebra antes' sem nunca ter rodado a segunda metade, e o bug de encoding só aparece em Windows — onde ninguém rodava. **Sandbox não é portão; isto é.**"

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest]

- uses: actions/checkout@v4
  with:
    fetch-depth: 0    # a varredura de segredo lê o HISTÓRICO; o padrão traria 1 commit
                      # e faria o portão anunciar cobertura que não teve

- run: python scripts/test_check.py      # quando este passo cair, o commit reintroduziu
                                         # um bug que já custou uma sessão

- run: python scripts/task.py check-all

env:
  PYTHONIOENCODING: utf-8
```

PROVA [.github/workflows/check.yml](#) — arquivo inteiro, 53 linhas, com o motivo de cada decisão em comentário.

Uma lacuna, declarada: a cobertura de plataforma é sobre *codepage* (cp1252). Não há teste sobre terminadores de linha (CRLF) nem sobre byte-order-mark. Isso foi verificado por busca no código e no arquivo de testes durante a preparação deste dossiê.

24. A varredura de segredo — e a lacuna que fica declarada

O repositório descreve a varredura como "uma rede de arrasto, não uma garantia". Esta seção mostra exatamente onde a rede pega e onde ela deixa passar.

As 11 famílias de padrão

O número foi conferido no código durante a preparação deste dossiê, e os 11 padrões foram executados contra uma amostra sintética: todos casam, cada um com o rótulo certo, sem colisão entre eles.

#	FAMÍLIA	EXEMPLO DO QUE ELA PEGA
1	chave ou segredo literal	api_key, secret_key, client_secret, access_token, aws_secret_access_key ...
2	senha literal	password, senha, passwd, pwd
3	senha em connection string	postgres://user:SENHA@host
4	access key AWS	AKIA + 16 caracteres
5	chave privada	-----BEGIN ... PRIVATE KEY-----
6	JWT	token em três segmentos separados por ponto
7	chave estilo Stripe	sk_live_..., rk_test_...
8	token estilo OpenAI	sk- + 20 ou mais caracteres
9	token GitHub	ghp_, gho_, ghu_, ghs_, ghr_
10	token Slack	xoxb-, xoxa-, xoxp- ...
11	token GitLab	glpat-

Os dois refinamentos que vieram da auditoria

A auditoria externa de 30/07/2026 mediu a versão anterior detectando **0 de 8** segredos plantados em formato realista. Duas causas, ambas corrigidas:

- **O filtro anti-falso-positivo era largo demais.** Ele descartava a *linha inteira* se ela contivesse `<algo>`, `xxx` ou a palavra "exemplo" — então um comentário como `# ver <ticket-4412>` no fim da linha desligava a checagem. Hoje o filtro é avaliado **só contra o trecho casado**, nunca contra a linha inteira.
- **Os padrões exigiam aspas.** O formato mais comum de vazamento no mundo — a linha de `.env` sem aspas, `API_KEY=sk_live_...` — não era coberto. Hoje o padrão aceita valor sem aspas, exigindo pelo menos um dígito.

PROVA docs/b_external_audit_report_260730_0900.md:53-120 (a medição original) · scripts/check.py:351-359 e :377 (as correções, com o motivo em comentário).

A LACUNA, DECLARADA EM VOZ ALTA

Das 11 famílias, apenas 2 são exercitadas por teste automatizado — a isca usa um único formato, que aciona a família de chave literal e a de padrão Stripe. As outras nove (senha literal, connection string, AWS, chave privada, JWT, OpenAI, GitHub, Slack, GitLab) funcionam, mas **se alguém quebrar um desses padrões amanhã, nenhum teste acusa.**

E o número "8 de 8, zero falsos positivos em 12 iscas" citado no README é **medição externa**: o próprio laudo declara, na linha 78, que os valores de teste com entropia real vivem fora do repositório. Não é falso — mas não é reproduzível a partir deste repositório, e isso está sendo dito aqui em vez de ser omitido.

PROVA Busca por AKIA, eyJ, ghp_, xox, glpat e PRIVATE KEY em test_check.py retorna **zero** ocorrências: o único segredo plantado é a constante ISCA (:67, :103, :713), no formato api_key=sk_live_..., que aciona duas famílias · docs/b_external_audit_report_260730_0900.md:78.

Por que isso está no dossiê. Porque o kit inteiro é construído sobre a doutrina "toda checagem tem isca", e nove formatos sem isca são o buraco mais visível na própria doutrina. Uma banca técnica vai encontrar isso se procurar; é melhor que encontre aqui, com o tamanho exato do problema e o caminho de correção óbvio — uma isca por família.

25. Onde o dinheiro vaza num projeto feito com IA

Antes de falar em economia, é preciso ser específico sobre a despesa. Existem quatro torneiras abertas, e três delas ninguém vê na fatura.

A TORNEIRA	COMO ELA DRENA	POR QUE NINGUÉM PERCEBE
1 · Contexto fixo	Tudo o que é carregado em <i>toda</i> sessão é pago em toda sessão. Um arquivo de 2.000 tokens relido 40 vezes custa 80.000 tokens sem produzir nenhuma linha.	Aparece diluído. Nunca há um momento em que alguém diga "isto custou caro"; custa um pouco, sempre.
2 · Regeneração	A IA devolve o arquivo inteiro em vez do trecho. Paga-se a saída inteira e a releitura na sessão seguinte — e o diff fica grande demais para revisar.	Parece produtividade. O output é maior, então parece que a sessão rendeu mais.
3 · Retrabalho evitável	Parser escrito sem ver a fonte real; schema modelado sobre payload imaginado. O custo não é a sessão errada — são as seis sessões seguintes consertando.	É contabilizado como "o problema era difícil", não como "faltou olhar antes".
4 · Re-litígio	A mesma decisão discutida pela quarta vez, porque não foi registrada com o motivo que a matou.	Cada rodada parece uma conversa nova e legítima.

A ASSIMETRIA QUE JUSTIFICA O PIPELINE

Das quatro torneiras, três não são atacáveis por um modelo melhor. Contexto fixo, retrabalho por falta de amostra e re-litígio são consequências de processo ausente. Trocar o modelo torna cada sessão um pouco melhor e não fecha nenhuma das três.

26. O que foi medido, e o peso de cada evidência

Três fontes, com pesos deliberadamente diferentes. Dizer qual é qual é parte da evidência — uma narrativa apresentada como medição destrói a credibilidade das medições reais.

FONTE	DATA	PESO	O QUE ELA É, E O QUE ELA NÃO É
Análise de uso no projeto SCB docs/a_scb_usage_analysis	22/07/2026	Medição	Tamanhos de arquivo contados, custos em tokens calculados, com as ressalvas contra o próprio projeto explicitadas. É daí que vêm as regras 1, 4, 6 e 7.
Auditoria externa adversarial docs/b_external_audit_report	30/07/2026	Medição hostil	Postura declarada: "tudo é considerado falho até prova executada". Deu 58/100 ao kit. Cada afirmação com comando rodado ou linha citada. Reprovou a varredura de segredo com 18/100 .
Avaliação de campo — TAP GO docs/c_field_evaluation_tapggo	13/08/2026	Medição pré-registrada	Critério de sucesso escrito, datado e hasheado antes de qualquer arquivo do projeto ser aberto. Os agentes que construíram o projeto não sabiam que era teste do kit.
Caso de referência SPO c_technical_docs/b_reference_case	—	Narrativa não verificada	Os números ("14 passagens", "84 achados") vêm de relato, sem relatório nem commit anexado. O próprio arquivo se declara assim. Use como lista de armadilhas plausíveis, não como aferição.

PROVA README.md:106-111 — a seção "Sobre as evidências deste kit" pesa explicitamente **duas** fontes (a análise de uso e o caso de referência); a auditoria externa e a avaliação de campo são declaradas noutros pontos do README — **:29** e **:27**, respectivamente.

O que a avaliação de campo mediu

Projeto medido: **TAP GO v2** — jogo de disputa de pênaltis, aplicação estática sem backend, 9 módulos, **46 commits em 4 dias**, 34 sessões registradas. Nove hipóteses, cada uma com sinal de funcionando e de não funcionando, e limiar numérico fixado de antemão.

	HIPÓTESE	NÚMERO MEDIDO
H1	a lista de rejeitados impede a re-proposta	6 rejeitados (15% das linhas vivas), nenhum reaparece
H2	o orçamento de contexto segura	contexto em 95% , registro em 91% — dentro, no teto
H3	o portão é real	passa no HEAD · hook instalado · 3 de 3 sabotagens reprovadas
H4	pergunta sobe em vez de ser chutada	11 questões do dono, 8 respondidas (73%)
H5	o QA acha e fecha	16 achados em 3 passagens, 4 dos 5 críticos fechados
H6	o commit cita o ID	45 de 46 = 98% (o limiar era 50%)
H7	delta, não regeneração	~0% — 3 candidatos, os 3 conferidos à mão, nenhum é reescrita
H8	escopo contido	12 de 13 commits de módulo no lugar certo · 24 recusas citando a regra de escopo
H9	as skills pagam o próprio custo	9 das 24 com rastro (limiar era 8) — 4 pela régua estrita

PROVA docs/c_field_evaluation_tapgo_260813.md:23-36 · o hash do critério pré-registrado está na linha 19.

A EVIDÊNCIA QUE NÃO É CIRCULAR

Um kit que manda escrever **D-NN** sempre vai mostrar **D-NN** — isso não prova nada. O que prova são quatro coisas que **não** são autocumpridas: as sabotagens do portão numa cópia descartável; o cruzamento formal que apanhou o contexto afirmando "suíte 220/220" sobre código que não estava no git; a taxa de regeneração medida no `git log`; e os defeitos deixados **abertos de propósito** pela regra de escopo — que são custo, não benefício, mas provam que a regra morde.

27. Os cinco mecanismos de economia, com o número de cada um

O pipeline não promete "usar menos IA". Ele ataca cinco desperdícios específicos, e cada um tem um número medido atrás.

Custo fixo por sessão, em tokens

O que é carregado em toda sessão, antes de qualquer trabalho começar. A fonte declara a tabela como **estimativa em tokens**, derivada de tamanhos de arquivo medidos no projeto SCB (tokens $\approx$ caracteres $\div$ 3 em português). Onde a fonte dá faixa, o gráfico usa o topo dela e a faixa está escrita ao lado.

■ Antes — kit v1, medido no projeto real ■ Depois — desenho atual

Instrução do papel (prompt ou skill) — a fonte dá faixa: 620–1.150 antes, 280–560 depois



Contexto-fonte



Registro de decisões — carregado inteiro nas sessões de evolução



Escala comum às três barras: 0 a 7.700 tokens. Valores rotulados na ponta de cada barra.

Os valores de "antes" derivam de tamanhos medidos nos arquivos reais do projeto SCB; os de "depois" são os tetos garantidos pelo desenho atual e cobrados por script. Usando o topo de cada faixa, uma sessão comum passa de $\sim$ 2.730 para $\sim$ 1.860 tokens de custo fixo; uma sessão de evolução, de $\sim$ 10.430 para $\sim$ 3.860. A conversão de caracteres em tokens é uma aproximação, e a fonte a declara como tal. Fonte: docs/a_scb_usage_analysis_260722_0000.md:45–51.

Mecanismo 1 · Orçamento de contexto em número, cobrado por script

O contexto-fonte tem teto de 4.000 caracteres, e o commit é reprovado se estourar. O que não cabe vai para um arquivo de tema, lido sob demanda — **nunca para prosa comprimida**. O número existe porque o adjetivo falhou: " $\leq$ 1 página" produziu um parágrafo de 1.930 caracteres relido em toda sessão.

O efeito colateral que também é economia: a auditoria mediu o template vazio do contexto em **3.541 de 4.000 caracteres** — ou seja, o kit entregava um projeto novo já com 88% do orçamento consumido por instruções. Foi reduzido para **2.677**. O projeto do usuário ganhou 864 caracteres de espaço útil que antes eram do kit.

PROVA docs/b_external_audit_report_260730_0900.md:18 (a tabela v4 $\rightarrow$ v5) · a checagem está em scripts/check.py:197.

Mecanismo 2 · Histórico e evidência fora do contexto

O changelog datado e os relatórios de QA existem, são consultáveis, e **nenhuma sessão os carrega**. No projeto SCB isso significou 173 linhas de histórico que ninguém pagou. E os relatórios de QA de um projeto real — *14 passagens, 84 achados*, número que vem do caso de referência e é **relato, não medição** — custaram zero em contexto e continuaram consultáveis.

PROVA e_qa/README.md, última seção · o contrato de leitura que proíbe varrer essas pastas está em CLAUDE.md:17 e :26.

Mecanismo 3 · Um mecanismo só — a fusão de prompts e skills

Na versão antiga havia prompts/ e skills/ separados, e a sessão carregava os dois. A sobreposição textual medida entre o prompt de QA e a skill de guardrails era de 27% — pagos duas vezes. Além disso, **6 de 6 prompts foram reescritos pelo próprio projeto** por serem prolixos: as versões reescritas faziam a mesma função por cerca de **45% menos**. Quando o usuário refaz 100% dos templates, o defeito é do template. Os papéis de fase viraram skills e a pasta prompts/ deixou de existir.

Mecanismo 4 · Delta em vez de regeneração

É o maior item, e o mais difícil de medir em tokens porque o custo real não é o output — é o diff que ninguém revisa e o código certo que volta diferente. O que *está* medido é a taxa: no projeto avaliado em campo, a regeneração ficou em ~0% — três candidatos levantados automaticamente, os três conferidos à mão, nenhum era reescrita.

Mecanismo 5 · Retrabalho evitado por "observe antes de construir"

Este é o de maior valor absoluto e o mais fácil de explicar para a banca funcional. Dois custos concretos já pagos, ambos por escrever código sobre uma estrutura imaginada:

- 6 ciclos de QA num projeto, resolvidos apenas quando se criou um modo `--investigar` para olhar a estrutura real primeiro;
- 6 versões de schema em 7 dias em outro, por não declarar as restrições do banco antes de modelar.

A regra virou pré-requisito de entrada em quatro skills: sem amostra real do payload, a sessão para e pede. E, quando não há amostra, a skill define o que entregar no lugar.

PROVA docs/a_scb_usage_analysis_260722_0000.md:35 (a saga dos escudos, QA-05 a QA-10) ·
b_process/d_agent_learnings.md, seção "Herdades".

28. O imposto: o que o pipeline cobra de volta

Nenhuma apresentação de processo é levada a sério se só mostra a economia. Esta seção existe para que a banca não precise descobrir o custo sozinha — e todos os números vêm da avaliação de campo, que os publicou junto com os benefícios.

O IMPOSTO	MEDIDO	O QUE SIGNIFICA NA PRÁTICA
Sessões gastas administrando o orçamento do próprio registro, sem uma linha de produto	15% 5 de 34	Uma sessão a cada sete é manutenção do processo. Este é o número mais importante desta página.
Commits que não tocam código	43%	Documentação, decisões e registro. Parte disso é o valor do kit; parte é peso.
Defeitos conhecidos e deixados abertos pela regra de escopo	9	Um deles era <i>uma linha de CSS</i> . A regra que impede consertar de carona também impede consertar o que é trivial.
Modo do produto parado esperando uma decisão do dono	4 dias	O canal de escalada funcionou — e o dono virou o gargalo. Havia ainda 2 questões com prazo estourado que o próprio kit registrou e não cobrou.
Retrabalho manual causado pelo teto de contexto medir formatação	1	Já corrigido no projeto.

PROVA docs/c_field_evaluation_tapgo_260813.md:55-63 — a seção literalmente chamada "O imposto, que fica declarado".

E o veredito, com o defeito do instrumento

Pela letra do critério pré-registrado, o resultado foi **ATRAPALHOU**: a cláusula dizia "qualquer teste de atrapalho disparado", e dois dispararam. Pela **evidência**, 9 de 9 hipóteses ficaram no lado positivo.

A régua estava errada, e isso ficou escrito — porque consertá-la em silêncio depois de ver o resultado é o que invalida uma avaliação inteira:

- **Um dos testes não tinha controle de taxa-base.** Um canal de escalada que funciona sempre terá uma questão aberta e bloqueante em qualquer foto instantânea. O teste transformava o funcionamento normal numa prova de fracasso; o certo seria medir a *taxa* (73%) e o tempo até a resposta.
- **O outro disparou num incidente que o kit detectou, mediu e corrigiu estruturalmente no mesmo dia.** O teste não distinguia "retrabalho induzido e absorvido" de "induzido e recorrente".

Com a régua consertada e o conserto declarado: AJUDOU, com imposto medido.

O QUE ESTA MEDIÇÃO NÃO PROVA — DITO ANTES QUE PERGUNTEM

n = 1. Um projeto, 4 dias, uma forma de aplicação, um dono. Ataca a falta de evidência; não a resolve.

Sem contrafactual. Não existe o mesmo projeto sem o kit. Nenhuma afirmação sobre "o agente não teria feito isto sozinho" pode passar de derivada.

Fora do alcance do agente: o projeto medido não tem teste automatizado de nenhuma tela. Isso é do dono e continua sendo.

PROVA docs/c_field_evaluation_tapgo_260813.md:82-89

29. Por que Obsidian, e por que nada depende dele

O repositório é, ao mesmo tempo, um repositório git e um cofre do Obsidian. Isso é de propósito — e a segunda metade da frase é a mais importante para a banca.

Nada no pipeline depende do Obsidian para funcionar. É tudo Markdown puro, os scripts são Python sem dependência, e as skills funcionam em qualquer ferramenta de IA. O Obsidian não é uma peça da arquitetura — é uma lente. Mas é uma lente que muda o custo de conduzir o processo, e vale explicar por quê.

O que o Obsidian resolve

RECURSO	O PROBLEMA QUE ELE ELIMINA
Links <code>[[assim]]</code>	O roteiro linka direto a skill de cada fase. Você não procura o arquivo: clica. Sem isso, cada troca de fase é uma busca no explorador de arquivos.
Backlinks	No rodapé de cada nota, quem aponta para ela. É como se descobre <i>todo lugar que cita uma decisão</i> — a pergunta que, sem essa lista, ninguém faz.
Templates	D-NN, QA-NN e fecho de sessão prontos para inserir com um atalho. O fecho de sessão é o passo mais pulado do pipeline; transformá-lo num clique é a diferença entre a regra existir e a regra acontecer.
Busca	Procurar por <code>D-</code> , <code>Q-</code> , <code>QA-</code> ou <code>T-</code> salta direto para o item rastreável. É o que torna os IDs úteis em vez de decorativos.
Grafo	Ver a estrutura de ligações do vault inteiro — o assunto da próxima seção.

PROVA `c_technical_docs/a_obsidian_guide.md:7` (a declaração de independência) e `:35–40` (a navegação).

A ARMADILHA QUE O GUIA AVISA, E O SCRIPT COBRA

A raiz do vault é a raiz do repositório git. Abrir a pasta-mãe faz os links de caminho pararem de resolver — e o `check.py` acusa, dizendo exatamente quais links quebraram. Não é um detalhe de conforto: é a checagem número 7, que reprova o commit.

PROVA `c_technical_docs/a_obsidian_guide.md:15` · a checagem em `scripts/check.py:304`.

Uma decisão de versionamento que merece nota

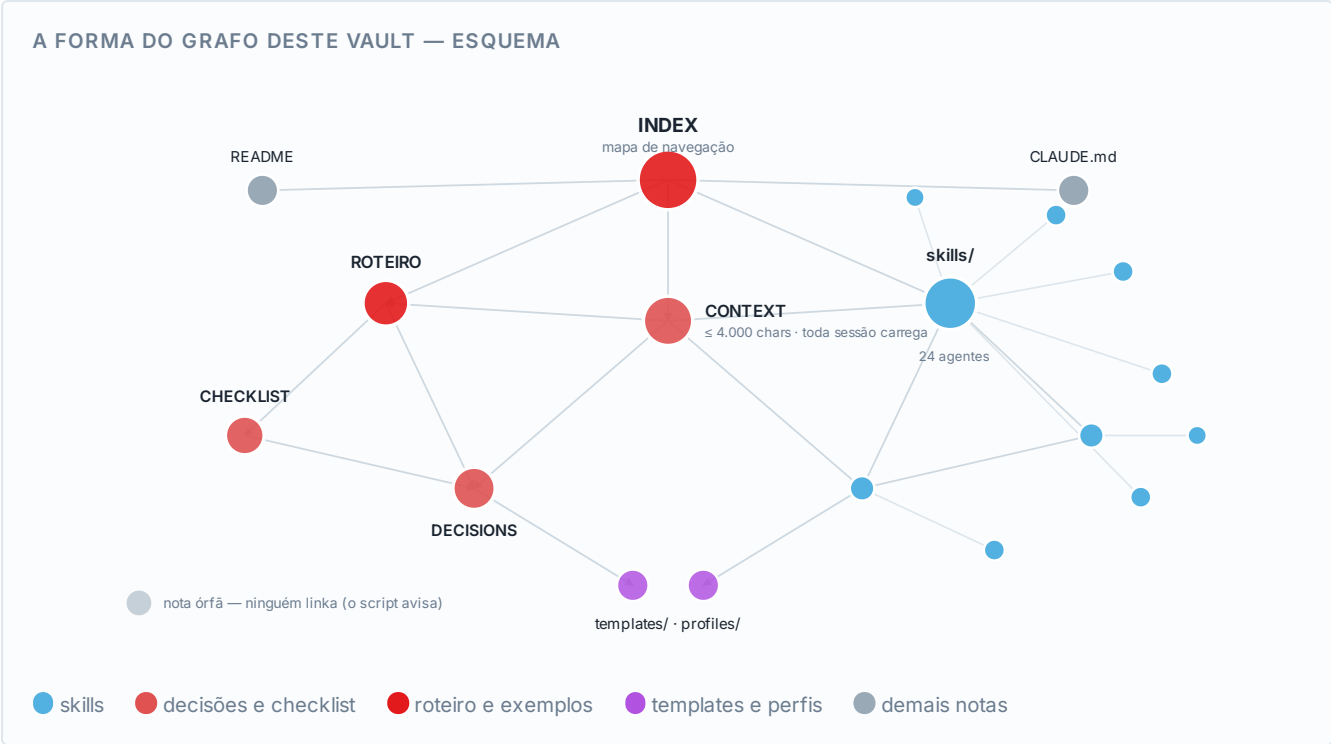
A pasta `.obsidian/` é versionada de propósito — é o que faz o vault abrir já configurado para quem clonar: nos favoritos certos, na nota inicial certa, com o grafo já colorido. A exceção é o arquivo de estado da janela, que muda a cada abertura e está no `.gitignore`. E a pasta inteira fica fora da validação do `check.py`, porque é configuração de ferramenta, não conteúdo do processo.

PROVA `c_technical_docs/a_obsidian_guide.md:76`

30. O grafo: o que ele mostra que uma lista de pastas não mostra

Uma árvore de arquivos mostra *onde as coisas estão*. O grafo mostra *o que conversa com o quê* — e é aí que os defeitos de estrutura aparecem sozinhos.

O grafo do Obsidian desenha uma bolinha por nota e uma linha por link. Notas muito referenciadas puxam as outras para perto e viram **hubs**; notas que ninguém referencia ficam soltas na borda. No caso deste repositório, isso não é enfeite: **a forma do grafo é uma verificação visual do desenho do processo.**



Esquema, não captura de tela: as posições foram desenhadas para caber na página, mas os hubs, os grupos e as cores são os configurados de fato em `.obsidian/graph.json`.

As três coisas que o grafo revela na hora

O QUE SE VÊ	O QUE SIGNIFICA	POR QUE IMPORTA
Três hubs, e só três INDEX, CONTEXT e skills/	A navegação converge para um mapa; o estado converge para um arquivo; os papéis convergem para um índice.	Se aparecesse um quarto hub inesperado, seria sinal de que algum documento virou fonte de verdade sem ninguém decidir isso. É a regra 6 (estado num lugar só) sendo verificada com o olho.
Notas soltas na borda	Nota que ninguém linka. O <code>check.py</code> emite um aviso — e o grafo mostra a mesma coisa sem ninguém rodar nada.	Documento que ninguém referencia não é lido. Ele envelhece, e um dia alguém o encontra e acredita nele.
O cacho azul das skills	Os 24 agentes formam um agrupamento denso, ligado ao resto por poucos pontos: o índice e o roteiro.	É a confirmação visual de "uma skill por sessão": cada agente é um nó independente que não depende dos outros 23 para funcionar.

Como o grafo está configurado, e o que cada opção significa

CONFIGURAÇÃO	EFEITO
<code>colorGroups</code> — 5 grupos	Skills, decisões/checklist, roteiro/exemplos, templates/perfis e um grupo herdado. As cores separam <i>função</i> , não pasta.
<code>hideUnresolved: true</code>	Link quebrado não vira nó fantasma. O grafo só mostra notas que existem — quem denuncia link quebrado é o script, não o desenho.
<code>showOrphans: true</code>	Notas sem nenhuma ligação aparecem . É deliberado: a órfã tem de ficar visível para ser resolvida.
<code>showArrow: true</code>	As ligações têm direção — dá para ver quem cita quem, não só que existe relação.
<code>linkDistance: 250</code> , <code>repelStrength: 10</code>	Nós bem afastados: os agrupamentos se separam visualmente em vez de virarem uma bola única.
<code>textFadeMultiplier: 0</code>	Os nomes das notas aparecem sempre, em qualquer nível de zoom. O grafo é para <i>ler</i> , não para admirar.

PROVA `.obsidian/graph.json` — arquivo inteiro, 58 linhas.

ACHADO ENCONTRADO AO PREPARAR ESTE DOSSIÊ

Dois dos cinco grupos de cor apontam para pastas que **não existem mais**: `path:prompts`, eliminada na refatoração "um mecanismo só" quando os papéis de fase viraram skills; e `path:perfis`, que virou `b_process/profiles/` quando o repositório adotou os nomes em inglês. Os dois grupos ficaram para trás e hoje não colorem nada. É inofensivo, mas é exatamente a espécie de resíduo que o kit chama de *cruft* — e está registrado aqui em vez de ser removido em silêncio. O guia do Obsidian, na linha 22, ainda lista "prompts" entre os grupos do grafo.

31. O que o vault já vem configurado para fazer

Clonar e abrir. Nenhum download, nenhum plugin da comunidade, nenhuma configuração manual.

- **Abre direto no INDEX**, o mapa de navegação — não numa nota aleatória.
- **Favoritos já na barra lateral**: Início, Roteiro, Contexto, Decisões, Backlog, Checklist, Skills, Templates — *com a ressalva do quadro abaixo*.
- **Plugins do núcleo ativos**: 21 no total, entre eles Grafo, Busca, Backlinks, Outline, Tags, Paleta de comandos, Pré-visualização, Templates e Propriedades. Todos nativos.
- **Templates apontando para a pasta certa** — D-NN, QA-NN e fecho de sessão prontos para inserir.
- **Grafo com cores por tema**, como mostrado na seção anterior.

PROVA `c_technical_docs/a_obsidian_guide.md:17-22` · `.obsidian/core-plugins.json` e `.obsidian/bookmarks.json`.

SEGUNDO ACHADO ENCONTRADO AO PREPARAR ESTE DOSSIÊ

Os 8 favoritos apontam para caminhos que **não existem mais**: `INICIO.md`, `ROTEIRO.md`, `CONTEXT.md`, `DECISIONS.md`, `BACKLOG.md`, `CHECKLIST.md`, `skills/LEIA-ME.md` e `templates/LEIA-ME.md` — os nomes de antes da v6-v7, que adotou o padrão de nomes da equipe. Nenhum resolve hoje. É a mesma espécie do grupo de cor morto, em escala 8x, e vem da mesma causa: a pasta `.obsidian/` fica fora da validação do `check.py`. A decisão de mantê-la fora é defensável — é configuração de ferramenta —, mas o preço dela está aqui, medido.

Os três templates, e por que o terceiro é o que mais importa

TEMPLATE	COLA EM	QUANDO
decisao	tabela de decisões	assunto fechado — inclusive rejeição
achado-qa	tabela de achados	cada achado de uma passagem de QA
fecho-de-sessao	nota do dia	toda vez que uma sessão termina

O guia recomenda atribuir um atalho de teclado ao terceiro, com uma justificativa que resume a Parte VII inteira: *o fecho de sessão é o passo mais pulado do pipeline, e é onde o estado começa a divergir*. A ferramenta não conserta a disciplina — ela reduz o custo de ter disciplina, e é assim que ela ganha.

PROVA `c_technical_docs/a_obsidian_guide.md:24-33`

UMA NOTA DE RIGOR QUE O PRÓPRIO GUIA FAZ

O guia do Obsidian **não lista** o que reprova e o que só avisa. A lista mora no cabeçalho do `check.py`, que é a fonte da verdade, com a justificativa escrita: *"lista repetida em documentação derrapa em silêncio quando o script muda, e divergência doc × código é achado de QA pela regra do próprio kit."* Este dossiê reproduziu essa lista (seção 20) — e o fez lendo o script, não a documentação, justamente por isso.

PROVA `c_technical_docs/a_obsidian_guide.md:72`

32. A linha do tempo: da v1 à v13.4 em 23 dias

24 commits, de 22 de julho a 13 de agosto de 2026. Mais importante que a velocidade é o padrão: **quase toda versão nasceu de uma medição, não de uma ideia.**

18 VERSÕES DO KIT da v1 à v13.4	23 DIAS 22/07 a 13/08/2026	3 AUDITORIAS interna · externa · de campo	16 ACHADOS QA FECHADOS QA-01 a QA-16, registrados no changelog do kit
VERSÃO	DATA	O QUE MOTIVOU	O QUE MUDOU
v1	—	Baseline	O kit original, 568 linhas. O que funcionou e o que falhou nele está medido na análise de uso.
v2	22/07	Medição de um projeto real	Orçamentos de contexto <i>em número</i> , estado em fonte única, prompts imperativos (~metade do custo), a regra "observe antes de construir", arquivo de aprendizados — e o nascimento do <code>check.py</code> .
v2.1	22/07	Generalização	Perfil genérico para qualquer stack, modo curto para projeto pequeno, arquivamento do registro de decisões.
v3	30/07	Faltava o caminho operável	12 agentes instaláveis , os portões de existência , o roteiro fase → skill → portão, o Obsidian versionado, os templates e o <code>new_project.py</code> .
v4	30/07	Auditoria interna adversarial	A pasta <code>prompts/</code> foi dissolvida (27% de sobreposição medida com as skills). O rigor deixou de ser prosa: a auditoria contou 8 de 163 itens verificados por máquina. Nasceram a varredura de segredo, o <code>install_hook.py</code> e a tabela "onde este kit para".
v5	30/07	Auditoria externa — placar 58/100	A v4 declarou ter fechado a lacuna de segurança; a auditoria mediu que não fechou . Seis correções críticas, cada uma com teste antes e depois. Nasceu o <code>CLAUDE.md</code> . Detalhe na próxima seção.
v6–v7	03/08	Padrão da equipe	O repositório adota o padrão de nomes em inglês com prefixo de ordem, e o kit passa a ser <i>exatamente</i> o que o padrão chama de pasta de documentação. Entra o glossário.
v8	05/08	Uso real em Windows pt-BR sob OneDrive	Cinco achados encontrados <i>usando</i> , não testando — incluindo os bugs de encoding que só existem fora do Linux.
v9	05/08	Comparação com alternativa	Quatro lacunas apontadas; três adotadas, uma rejeitada com o motivo registrado.
v10	05/08	<code>QA-06</code>	O <code>--upgrade</code> passa a preservar customização do dono; nasce a cobertura módulo × tarefa.
v11	05/08	<code>QA-07</code> — parser silencioso	Nasce o canário dos templates : uma edição cosmética num template fazia a checagem parar de checar, em silêncio.
v12	05/08	Escopo vazando	Seção <i>Limites</i> obrigatória em toda skill, com o esquema cobrado por script.
v13	05/08	<code>QA-08</code> — honestidade	A frase de auto-cobertura do README passa a ser recontada por teste . Ela dizia 188/18 com 277/23 no disco.
v13.1	05/08	<code>QA-09 ... QA-13</code>	Deriva de documentação × código, e projeto novo nascendo limpo.
v13.2	13/08	Avaliação de campo — <code>QA-14</code>	A checagem de ID enxergava 12% do que dizia enxergar. Nasce a regra <i>uma isca por checagem</i> , cobrada por teste.

VERSÃO	DATA	O QUE MOTIVOU	O QUE MUDOU
v13.3	13/08	QA-15 + 5 melhorias	Um <code>.gitignore</code> só com comentários passava na checagem. Entram: número declarado conferido, fila do dono visível, achado vencido, e o <code>arquivar.py</code> .
v13.4	13/08	QA-16	A correção do QA-14 teria feito 22 IDs legitimamente arquivados virarem fantasmas — e o portão reprovaria todo commit de um projeto real.

PROVA `d_history/b_kit_changelog.md` — 535 linhas, uma seção por versão, cada mudança com o número que a motivou · `git log`: 24 commits, autor único, 2026-07-22 a 2026-08-13.

O PADRÃO QUE A LINHA DO TEMPO REVELA

Das 18 versões, **onze nasceram de um defeito medido** — três auditorias e oito achados de QA numerados. Nenhuma nasceu de "seria bom se". Esse é o argumento mais forte a fazer na banca: o kit não foi projetado e depois validado; ele foi **corrigido até parar de mentir**, e cada correção deixou um teste para trás.

33. As correções que só apareceram porque foram medidas

Quatro casos em que o kit *declarava* fazer uma coisa e *não fazia*. Todos foram encontrados por medição, nenhum por leitura. É a seção que mais convence uma banca técnica.

Caso 1 · A flag que varria zero commits

A Fase 6 depende de `--historico-completo` para varrer o histórico inteiro atrás de segredos. O código montava um comando que, na linguagem do git, significa **nenhum commit** — não "todos". A flag era uma operação vazia que imprimia OK.

O sintoma que denunciou é a parte bonita: a varredura "completa" rodava em 0,33 s contra 1,49 s da parcial. *A varredura completa era mais rápida porque não varria nada.*

Caso 2 · O scanner que detectava 0 de 8 segredos

A versão 4 declarou ter fechado a lacuna de segurança. A auditoria externa plantou oito segredos em formato realista e mediu: **zero detectados**, saída verde, commit liberado. Duas causas — um filtro anti-falso-positivo que desligava a linha inteira se ela contivesse um comentário, e padrões que exigiam aspas, deixando passar o formato mais comum de vazamento no mundo. Depois da correção: **8 de 8**, com zero falso-positivo em 12 iscas.

Caso 3 · A checagem que enxergava 12%

Já detalhado na seção 22. O ponto que vale repetir aqui: **já existia um canário para essa checagem, e ele passava** — porque escrevia o ID sem crases, do jeito que ninguém escreve. A cegueira sobreviveu a uma auditoria externa que citou o portão como ponto forte.

Caso 4 · O conserto validado no caso mais fácil

A correção do caso 3 foi validada contra o *template* do kit — que não tem arquivo-morto de decisões. Ao medir o merge antes de fazê-lo, num projeto real, o número apareceu: 0 fantasmas virariam 22, e o portão passaria a reprovar todo commit. A lição ficou escrita: *conserto validado só no template é conserto validado no caso mais fácil.*

A ESPÉCIE, NÃO O CASO

O que liga os quatro: em todos, a resposta não foi consertar o caso. Foi **consertar a espécie** — criar um teste que cobre a classe inteira de defeitos. O caso 3 gerou a regra "toda checagem numerada tem isca"; o caso 4 gerou a regra "rode contra o projeto real antes de chamar de pronto". É por isso que os testes cresceram de 23 para 57 em uma semana.

PROVA `d_history/b_kit_changelog.md:11-38` (v13.4), `:98-147` (v13.2), `:480-496` (v5) · `b_process/d_agent_learnings.md`, seção "Herdadas".

E sete achados encontrados na preparação deste dossiê

Preparar esta apresentação foi, na prática, mais uma passagem de revisão — e ela rendeu sete divergências. Estão listadas aqui, com a linha exata, em vez de terem sido corrigidas em silêncio, porque é essa a regra do próprio kit. **Seis das sete são da mesma espécie:** número ou caminho mantido à mão, numa cópia que nenhum teste alcança.

ONDE	O QUE ESTÁ ERRADO
1 <code>README.md:72</code>	Diz "o portão de higiene (14 falhas · 12 avisos)". O código tem 16 avisos . Ironia registrada: a linha 25 do mesmo arquivo diz 16 e <i>está certa</i> — porque é ela que o teste de honestidade cobre. A linha 72 está num bloco de árvore de diretórios, fora do alcance do teste.
2 <code>b_process/f_glossary_and_primer.md:211</code>	Diz "São 23 " agentes. São 24 — contados em disco. O mesmo arquivo diz 24 na linha 310, e todos os outros índices dizem 24.
3 <code>b_process/skills/data-analysis/SKILL.md</code>	Duas seções com o título " <i>Armadilhas pagas</i> " (linhas 59 e 67), com conteúdo parcialmente sobreposto. Nenhuma das outras 23 skills repete cabeçalho.
4 <code>.obsidian/graph.json</code>	Dois grupos de cor apontam para pastas que não existem: <code>path:prompts</code> (eliminada na v4) e <code>path:perfis</code> (renomeada para <code>profiles/</code> na v6–v7). Nenhum dos dois colore nada. <code>a_obsidian_guide.md:22</code> ainda lista "prompts".
5 <code>.pytest_cache/v/cache/nodeids</code>	Cache obsoleto versionado: lista 23 testes em 8 classes; o arquivo atual tem 57 em 18. Quem abrir a pasta subestima a suíte em ~60%.
6 <code>b_process/f_glossary_and_primer.md:42-43</code>	O achado mais interessante dos seis. A frase diz "o kit tem 188 itens de checklist; o script julga 18 (~10%)" — exatamente o par de números obsoletos que a v13 corrigiu e passou a cobrar por teste. O teste de honestidade só lê o <code>README.md</code> ; esta cópia da mesma afirmação, noutro arquivo, escapou dele. É a prova viva de que a correção da v13 tratou o caso e não a espécie.
7 <code>.obsidian/bookmarks.json</code>	Os 8 favoritos da barra lateral apontam para caminhos que não existem mais — <code>INICIO.md</code> , <code>ROTEIRO.md</code> , <code>CONTEXT.md</code> , <code>DECISIONS.md</code> , <code>BACKLOG.md</code> , <code>CHECKLIST.md</code> , <code>skills/LEIA-ME.md</code> , <code>templates/LEIA-ME.md</code> . São os nomes anteriores à v6–v7, que renomeou tudo para o padrão da equipe. Mesma espécie do achado 4, em escala 8×.

Os sete são cosméticos — nenhum afeta o comportamento do portão, e o kit continua passando na própria suíte (57 testes verdes) e no próprio `check-all`. Mas o achado 6 é o que vale levar para a banca: ele mostra a doutrina do kit sendo aplicada ao próprio kit e encontrando o ponto em que ela ainda não foi longe o bastante. O caminho de correção é o do kit — não trocar o número, e sim estender o teste de honestidade para cobrir toda cópia da afirmação, não só a do README.

34. Onde o pipeline para

A tabela abaixo está no README desde a versão 4, e a auditoria externa a apontou como **o ponto mais forte do kit**. Ela é o motivo de esta apresentação poder ser feita sem medo de pergunta.

CONTEXTO	SERVE?	O QUE TRAVA
App pequeno ou médio, um dono	Sim — é o alvo	Nada. Se for pequeno, use o modo curto.
Projeto que já existe	Sim, desde a v5	Comece pela skill de adoção, que produz contexto e plano a partir do código real. Quanto maior o sistema, mais o teto de 4.000 caracteres aperta.
Sustentar o que já roda	Sim, desde a v5	As cinco skills de sistema vivo cobrem adoção, depuração, performance, observabilidade e dependências.
Time de 2 a 4 pessoas	Parcialmente	O limite de trabalho em paralelo é declarado e cobrado — basta subi-lo. Mas não há atribuição por pessoa nem merge de decisões concorrentes.
App grande (30+ módulos)	Não	Um contexto de 4.000 caracteres não representa 30 módulos. Solução parcial: uma lista de módulos em arquivo separado, com o contexto guardando só a contagem.
Projeto longo (100+ decisões)	Com atrito	O teto de 12.000 caracteres do registro dá cerca de 66 linhas; o arquivamento é manual , e ninguém lembra.
Multi-repo ou monorepo grande	Não	O kit assume um repositório e um contexto.
CI/CD, revisão por pares	Não cobre	O único automatismo é o pre-commit do <code>check.py</code> .

PROVA `README.md:14-23`

A LIMITAÇÃO MAIS IMPORTANTE, DITA COM O NÚMERO

São 284 itens de checklist e o script julga 30 — cerca de 11%. O resto depende de alguém rodar a seção certa. **Isto é um kit de disciplina com algumas travas automáticas, não um sistema que impede erro.**

E essa frase é cobrada por teste, porque a frase mais honesta do kit não pode ser a que envelhece em silêncio — ela já envelheceu uma vez.

35. As perguntas que vão ser feitas, com a resposta pronta

Quinze perguntas prováveis, separadas por quem tende a fazê-las. A resposta curta está em negrito; a linha seguinte é o número ou a prova, para o caso de a pergunta continuar.

Da banca funcional

"Isso não é burocracia demais para usar um assistente de código?"

O ciclo inteiro são sete passos, e a maior parte é uma linha de texto. O que custa tempo é o portão — e ele custa exatamente o tempo que você gastaria descobrindo o defeito depois, com juro. Para projeto pequeno existe o modo curto: fases 0, 2, 4 e 6. *Pula-se fase, não se pula regra.*

"Quanto tempo custa adotar num projeto que já existe?"

Uma sessão. A skill de adoção lê o *código* — não a documentação — e produz o contexto, o plano retroativo e a lista do que ninguém soube explicar. Ela não altera nenhuma linha de código, e isso é portão verificável por `git diff`.

"O que acontece se alguém esquecer um passo?"

Boa parte é cobrada no commit: orçamento estourado, estado divergente, ID citado que não existe, segredo versionado, link quebrado. O que o script não cobre está na checklist — e essa parte é do dono. São 30 checagens automáticas para 284 itens.

"Isso funciona com qual ferramenta de IA?"

Qualquer uma. É Markdown puro e Python sem dependência. As skills podem ser instaladas por pasta ou simplesmente coladas na sessão. Não há API, nem chave, nem serviço.

"E se a pessoa que criou isso sair da equipe?"

Está tudo escrito, e o repositório é a fonte. O README carrega os porquês, o roteiro carrega o caminho, o glossário carrega o vocabulário, e o changelog do kit carrega o motivo de cada versão. Nenhuma regra depende de alguém lembrar de explicá-la.

Da banca técnica

"O portão automático pega tudo?"

Não, e isso está medido: 30 checagens para 284 itens de checklist — cerca de 11%. Quem afirma que o processo impede erro está vendendo. O que ele faz é tornar o erro visível e rastreável.

"Como vocês sabem que o próprio portão funciona?"

Uma isca por checagem numerada, e um teste que reprova se alguém adicionar uma checagem sem isca. São 14 iscas, cada uma plantando um problema realista e exigindo o código de retorno e o trecho de mensagem certo. Mais uma contraprova: o kit entregue, sem isca, tem de passar.

"Por que 57 testes num repositório de documentação?"

Porque uma checagem que para de checar não grita — continua imprimindo verde. Já aconteceu: uma checagem enxergava 12% do que dizia enxergar e sobreviveu a uma auditoria externa que a citou como ponto forte.

"Por que o CI roda em Windows?"

Porque os dois bugs de encoding que o kit já pagou não reproduzem no Linux. Um deles fez o portão nunca rodar na máquina do dono — todo "OK" vinha do sandbox Linux do agente. Sandbox não é portão.

"A varredura de segredo é confiável?"

É uma rede de arrasto, não uma garantia — e o repositório usa exatamente essas palavras. São 11 famílias de padrão, medidas contra 8 formatos reais (8 de 8, zero falso-positivo em 12 iscas). Mas só 2 das 11 famílias têm isca automatizada, e o número 8/8 é medição externa não reproduzível a partir deste repositório. Está na seção 24, com o tamanho exato do problema.

"Vocês não estão só descrevendo boas práticas conhecidas?"

A diferença é que cada regra aqui tem um custo pago atrás e uma trava na frente. "Observe antes de construir" é uma boa prática conhecida; "6 ciclos de QA e 6 versões de schema" é o preço que este projeto pagou por ignorá-la, e "sem amostra real a sessão para e pede" é a trava. Boa prática sem trava é decoração.

"Por que não usar uma ferramenta pronta de gestão?"

Porque o problema não é gestão de tarefas. O kit não tem estimativa, sprint, atribuição nem velocity — e diz isso explicitamente. O problema é o custo de contexto e a ausência de portão numa conversa com uma IA, e nenhuma ferramenta de gestão toca nisso.

Da banca cética

"Qual a evidência de que isso funciona?"

Uma medição de campo, com critério pré-registrado e hashado antes dos dados. Nove hipóteses de nove no lado positivo: 98% de commits com ID, ~0% de regeneração, 3 de 3 sabotagens do portão reprovadas. E é $n = 1$. Ataca a falta de evidência; não a resolve.

"O que isso custa?"

15% das sessões foram administrar o orçamento do próprio kit, sem uma linha de produto. Mais: 43% dos commits não tocam código, 9 defeitos ficaram abertos pela regra de escopo (um deles de uma linha de CSS), e um modo do produto ficou 4 dias parado esperando uma decisão do dono. Está tudo publicado junto com os benefícios, na mesma página.

"Vocês encontraram algum problema preparando esta apresentação?"

Sete. Estão na seção 33, com arquivo e linha. Seis dos sete são a espécie que o próprio kit batizou: número ou caminho mantido à mão, numa cópia que nenhum teste alcança. O melhor deles é o sexto: uma frase noutro arquivo ainda diz "188 itens, o script julga 18" — os números obsoletos que a versão 13 corrigiu e passou a cobrar por teste, mas só dentro do README. Nenhum afeta o comportamento do portão, e o caminho de correção é o do kit: estender o teste, não trocar o número.

SE A APRESENTAÇÃO TIVER DE TERMINAR EM UMA FRASE

Este pipeline não deixa a IA mais inteligente. Ele torna caro errar em silêncio — e essa é a única coisa que nenhum modelo melhor resolve sozinho.

36. Índice de provas

Os arquivos que sustentam as afirmações deste dossiê, com o que cada um prova. A ordem é a do

repositório.

ARQUIVO	TAM.	O QUE ELE PROVA NESTE DOSSIÊ
<code>README.md</code>	11 KB	As 7 regras com o porquê · a tabela "onde este kit para" · a frase de cobertura 284/30/11% · o papel do dono e as frases de segurança
<code>INDEX.md</code>	5 KB	O mapa do vault · a lista dos 24 agentes por grupo · a justificativa do CI em duas plataformas
<code>CLAUDE.md</code>	3 KB	O contrato de leitura: o que carregar, em que ordem, e o que nunca ler
<code>a_context/a_context_source.md</code>	3 KB	O template do contexto-fonte, com o formato fixo de "Estado atual" em 6 bullets
<code>b_process/a_roadmap.md</code>	13 KB	As 7 fases com skill e portão · o critério de saída do laço de QA · o ritmo esperado de 10 dias
<code>b_process/b_checklist.md</code>	11 KB	Os 118 itens de portão por tipo de entrega
<code>b_process/c_backlog.md</code>	2 KB	O WIP declarado · a justificativa do campo <code>**Módulo:**</code> · a lane de faxina
<code>b_process/d_agent_learnings.md</code>	4 KB	As 22 lições herdadas, incluindo "isca tem de sabotar como a casa escreve"
<code>b_process/f_glossary_and_primer.md</code>	16 KB	Os 6 modos de falha com custo · o glossário · o ciclo de 7 passos · quem faz o quê
<code>b_process/skills/README.md</code>	9 KB	Os 24 agentes agrupados, com o que cada um protege · a ordem de combinação
<code>b_process/skills/*/SKILL.md</code>	24 arq.	Os 166 itens de portão · as descriptions literais · os portões de existência · os limites de cada agente
<code>c_technical_docs/a_obsidian_guide.md</code>	6 KB	A independência em relação ao Obsidian · os templates · a decisão de versionar <code>.obsidian/</code>
<code>d_history/b_kit_changelog.md</code>	53 KB	As 18 versões, cada uma com o defeito medido que a motivou
<code>docs/a_scb_usage_analysis...</code>	9 KB	Medição: os números de contexto e token · as 7 invenções que viraram regra · a tabela de custo v1 → v2
<code>docs/b_external_audit_report...</code>	24 KB	Medição hostil: placar 58/100 · 0 de 8 segredos detectados · a flag que varria zero commits · 12,3 s → 1,5 s
<code>docs/c_field_evaluation_tapgo...</code>	6 KB	Medição pré-registrada: as 9 hipóteses · o imposto de 15% · o defeito do próprio instrumento
<code>e_qa/README.md</code>	2 KB	Por que os relatórios ficam fora do contexto — 14 passagens e 84 achados a custo zero
<code>scripts/check.py</code>	718 li.	As 14 checagens que reprovam e as 16 que avisam · as 11 famílias de segredo · os comentários que documentam cada bug pago
<code>scripts/test_check.py</code>	1.051 li.	Os 57 testes em 18 classes · as 14 iscas · os 4 testes de encoding · o teste de honestidade declarada
<code>scripts/task.py</code>	80 li.	O ponto de entrada único, e a justificativa de não ser um Makefile
<code>.github/workflows/check.yml</code>	53 li.	A matriz Linux + Windows · o <code>fetch-depth: 0</code> · "sandbox não é portão; isto é"
<code>scripts/new_project.py</code>	482 li.	A criação de projeto novo e o <code>--upgrade</code> que preserva a customização do dono
<code>scripts/install_hook.py</code> · <code>arquivar.py</code>	139 · 133	O pre-commit que torna o portão padrão · o arquivamento que relata antes de escrever

ARQUIVO	TAM. O QUE ELE PROVA NESTE DOSSIÊ
c_technical_docs/b_reference_case_spo.md	6 KB O caso de referência — e a própria declaração de que é narrativa, não medição
b_process/templates/	4 arq. Os modelos de D-NN, QA-NN e fecho de sessão
.obsidian/graph.json	58 li. Os 5 grupos de cor do grafo e as opções de exibição
.obsidian/core-plugins.json · bookmarks.json	— Os 21 plugins do núcleo ativos · os 8 favoritos (e os 8 caminhos mortos da seção 33)
.pytest_cache/v/cache/nodeids	— O cache obsoleto: 23 testes em 8 classes contra os 57 em 18 de hoje

Comandos para conferir na hora, se a banca pedir

```
python scripts/task.py --help      # todas as tarefas do kit, num lugar só
python scripts/task.py check      # o portão: orçamento, links, IDs, segredo (30 commits)
python scripts/task.py check-all # igual, varrendo o histórico INTEIRO – use antes de entregar
python scripts/task.py test       # os 57 testes de regressão dos próprios scripts
python scripts/task.py strict     # avisos também reprovam
python scripts/task.py hook       # instala o pre-commit
python scripts/task.py arquivar   # lista decisões que nenhum documento vivo cita
python scripts/task.py unhook     # remove o pre-commit deste kit
```

Dossiê preparado a partir do repositório `pipeline-projetos-IA` na versão do kit **13.4** · 24 commits, de 22/07/2026 a 13/08/2026 · todos os números foram recontados nos arquivos ou reproduzidos rodando o código.